

[REDACTED]

[REDACTED]

**Network Security Group Coursework
2025-26 Academic Year 1**

Group number: 73

Group size: 4

Group member 1: [REDACTED]

Group member 2: [REDACTED]

Group member 3: [REDACTED]

Group member 4: Mark Farrugia

Report word count: 1184

Table of Contents

Level 1 - Network setup and Connectivity.	3
Network Topology	3
Mininet Setup	3
Connectivity Testing.....	4
Level 2 - Traffic Generation and Analysis.....	7
Generation of Traffic	7
Set the Server to capture traffic.....	7
Generate TCP traffic.....	7
Generate UDP Traffic.....	7
Kill dump.....	7
Wireshark.....	8
General Traffic.....	8
Protocol Hierarchy.....	10
Wireshark Layer Inspection	11
Graph Inspection.....	14
Level 3 - Simulated Network Attack	15
Server-Side Setup for ICMP Flood Attack Capture.....	15
Attack 1: ICMP Flood (Ping Flood).....	16
Generate Baseline TCP Traffic	16
Launch ICMP flood	16
Inspect Traffic:	17
Attack 2: TCP SYN Flood	19
Launch SYN flood.....	19
Start normal traffic.....	19
Launch SYN flood from PC3 targeting Server port 5001	19
SYN Flood with IP Spoofing	20
Inspect in Wireshark.....	21
Level 4 - Network Defence with Firewall	25
Defend ICMP	25
Enable Firewall to block ICMP Requests	25
Perform ICMP Attack.....	26
Verification of Firewall - ICMP	28
Flush Previous Rules.....	28
Firewall to Block Router-Forwarded packets from PC3 -> Server.....	29
Initiate Attack.....	30
Analyse Traffic and Verify Firewalls	33
Comparison Between Levels	34
Task 5 - Critical evaluation and reflection.....	35

Level 1 - Network setup and Connectivity.

Network Topology

Figure 1.1.1 shows the network setup. A routed “extended star” topology.

Extended Star Topology Network

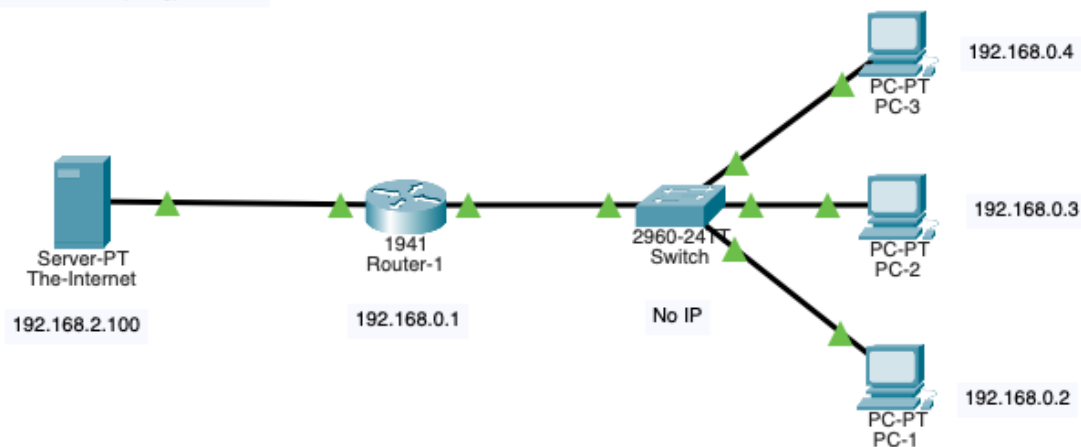


Figure 1.1.1 - Network Topology.

Mininet Setup

In Mininet (Figure 1.1.2), Figure 1.1.1 was replicated. This included three client hosts (PC1 192.168.0.2, PC2 192.168.0.3, PC3 192.168.0.4) connected via a switch to Router1 (LAN interface 192.168.0.1) and a separate server segment hosting Server 192.168.2.100.

```
parallels@ubuntu-gnu-linux-24-04-3:~$ sudo python3 coursework-1.py
*** Adding controller
*** Add switches
*** Add router
*** Add hosts
*** Add links
*** Starting network
*** Configuring hosts
Router1 Server PC1 PC2 PC3
*** Starting controllers
*** Starting switches
*** Configuring router
*** Configuring NAT for internet access
net.ipv4.ip_forward = 1
*** Network ready
*** Extended Star Topology Network
*** Server = "The-Internet" (192.168.2.100)
*** Test with: PC1 ping PC2 | PC1 ping PC3 | PC1 ping Server
*** Starting CLI:
mininet> net
Router1 Router1-eth0:Switch1-eth4 Router1-eth1:Server-eth0
Server Server-eth0:Router1-eth1
PC1 PC1-eth0:Switch1-eth1
PC2 PC2-eth0:Switch1-eth2
PC3 PC3-eth0:Switch1-eth3
Switch1 lo: Switch1-eth1:PC1-eth0 Switch1-eth2:PC2-eth0 Switch1-eth3:PC3-eth0 Switch1-eth4:Router1-eth0
Controller
mininet> dump
<Node Router1: Router1-eth0:0.0.0.0,Router1-eth1:None pid=29690>
<Host Server: Server-eth0:192.168.2.100 pid=29692>
<Host PC1: PC1-eth0:192.168.0.2 pid=29694>
<Host PC2: PC2-eth0:192.168.0.3 pid=29696>
<Host PC3: PC3-eth0:192.168.0.4 pid=29698>
<OVSSwitch Switch1: lo:127.0.0.1,Switch1-eth1:None,Switch1-eth2:None,Switch1-eth3:None,Switch1-eth4:None pid=29685>
<OVSController Controller: 127.0.0.1:6633 pid=29677>
mininet>
```

Figure 1.1.2 – Mininet Proof of work and layout of network proving Packet Tracer in Mininet.

Connectivity Testing

Figures 1.2.1, 1.2.2, and 1.2.3 show that connectivity was validated using ping between hosts and the server, demonstrating end-to-end IP reachability across subnets (e.g., PC1↔PC2/PC3 and PC1→Server).

```
mininet> PC1 ping PC2 -c 3
PING 192.168.0.3 (192.168.0.3) 56(84) bytes of data.
64 bytes from 192.168.0.3: icmp_seq=1 ttl=64 time=2.93 ms
64 bytes from 192.168.0.3: icmp_seq=2 ttl=64 time=2.42 ms
64 bytes from 192.168.0.3: icmp_seq=3 ttl=64 time=0.282 ms

--- 192.168.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 0.282/1.877/2.929/1.146 ms
mininet> PC1 ping PC3 -c 3
PING 192.168.0.4 (192.168.0.4) 56(84) bytes of data.
64 bytes from 192.168.0.4: icmp_seq=1 ttl=64 time=1.77 ms
64 bytes from 192.168.0.4: icmp_seq=2 ttl=64 time=0.477 ms
64 bytes from 192.168.0.4: icmp_seq=3 ttl=64 time=0.267 ms

--- 192.168.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 0.267/0.837/1.768/0.663 ms
mininet> PC1 ping Router1 -c 3
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=2.17 ms
64 bytes from 192.168.0.1: icmp_seq=2 ttl=64 time=0.654 ms
64 bytes from 192.168.0.1: icmp_seq=3 ttl=64 time=0.140 ms

--- 192.168.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2062ms
rtt min/avg/max/mdev = 0.140/0.989/2.174/0.863 ms
mininet> PC1 ping Server -c 3
PING 192.168.2.100 (192.168.2.100) 56(84) bytes of data.
64 bytes from 192.168.2.100: icmp_seq=1 ttl=63 time=2.20 ms
64 bytes from 192.168.2.100: icmp_seq=2 ttl=63 time=1.89 ms
64 bytes from 192.168.2.100: icmp_seq=3 ttl=63 time=0.308 ms

--- 192.168.2.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.308/1.464/2.200/0.827 ms
```

Figure 1.2.1 – PC1 pings.

```
mininet> PC2 ping PC1 -c 3
PING 192.168.0.2 (192.168.0.2) 56(84) bytes of data.
64 bytes from 192.168.0.2: icmp_seq=1 ttl=64 time=1.64 ms
64 bytes from 192.168.0.2: icmp_seq=2 ttl=64 time=0.128 ms
64 bytes from 192.168.0.2: icmp_seq=3 ttl=64 time=0.114 ms

--- 192.168.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2009ms
rtt min/avg/max/mdev = 0.114/0.626/1.636/0.714 ms
mininet> PC2 ping PC3 -c 3
PING 192.168.0.4 (192.168.0.4) 56(84) bytes of data.
64 bytes from 192.168.0.4: icmp_seq=1 ttl=64 time=2.26 ms
64 bytes from 192.168.0.4: icmp_seq=2 ttl=64 time=2.84 ms
64 bytes from 192.168.0.4: icmp_seq=3 ttl=64 time=0.239 ms

--- 192.168.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2010ms
rtt min/avg/max/mdev = 0.239/1.781/2.842/1.116 ms
mininet> PC2 ping Router1 -c 3
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=1.89 ms
64 bytes from 192.168.0.1: icmp_seq=2 ttl=64 time=2.84 ms
64 bytes from 192.168.0.1: icmp_seq=3 ttl=64 time=0.250 ms

--- 192.168.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.250/1.661/2.841/1.070 ms
mininet> PC2 ping Server -c 3
PING 192.168.2.100 (192.168.2.100) 56(84) bytes of data.
64 bytes from 192.168.2.100: icmp_seq=1 ttl=63 time=1.55 ms
64 bytes from 192.168.2.100: icmp_seq=2 ttl=63 time=2.76 ms
64 bytes from 192.168.2.100: icmp_seq=3 ttl=63 time=0.093 ms

--- 192.168.2.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2007ms
rtt min/avg/max/mdev = 0.093/1.465/2.758/1.089 ms
```

Figure 1.2.2 – PC2 pings.

```
mininet> PC3 ping PC1 -c 3
PING 192.168.0.2 (192.168.0.2) 56(84) bytes of data.
64 bytes from 192.168.0.2: icmp_seq=1 ttl=64 time=2.61 ms
64 bytes from 192.168.0.2: icmp_seq=2 ttl=64 time=1.30 ms
64 bytes from 192.168.0.2: icmp_seq=3 ttl=64 time=0.246 ms

--- 192.168.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2010ms
rtt min/avg/max/mdev = 0.246/1.385/2.614/0.968 ms
mininet> PC3 ping PC2 -c 3
PING 192.168.0.3 (192.168.0.3) 56(84) bytes of data.
64 bytes from 192.168.0.3: icmp_seq=1 ttl=64 time=0.691 ms
64 bytes from 192.168.0.3: icmp_seq=2 ttl=64 time=0.233 ms
64 bytes from 192.168.0.3: icmp_seq=3 ttl=64 time=0.308 ms

--- 192.168.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2043ms
rtt min/avg/max/mdev = 0.233/0.410/0.691/0.200 ms
mininet> PC3 ping Router1 -c 3
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=1.72 ms
64 bytes from 192.168.0.1: icmp_seq=2 ttl=64 time=1.42 ms
64 bytes from 192.168.0.1: icmp_seq=3 ttl=64 time=0.088 ms

--- 192.168.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 0.088/1.074/1.720/0.708 ms
mininet> PC3 ping Server -c 3
PING 192.168.2.100 (192.168.2.100) 56(84) bytes of data.
64 bytes from 192.168.2.100: icmp_seq=1 ttl=63 time=0.731 ms
64 bytes from 192.168.2.100: icmp_seq=2 ttl=63 time=0.490 ms
64 bytes from 192.168.2.100: icmp_seq=3 ttl=63 time=0.287 ms

--- 192.168.2.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2055ms
rtt min/avg/max/mdev = 0.287/0.502/0.731/0.181 ms
```

Figure 1.2.3 – PC3 Pings.

Level 2 - Traffic Generation and Analysis

Baseline traffic was generated using TCP and UDP throughput tests (iPerf), while packets were captured on the server interface with tcpdump and analysed in Wireshark at the protocol and flow levels.

Generation of Traffic

Set the Server to capture traffic

```
mininet> Server tcpdump -i Server-eth0 -w /tmp/baseline_traffic.pcap &
```

Generate TCP traffic

```
mininet> Server iperf -s -p 5001 &  
mininet> PC1 iperf -c 192.168.2.100 -p 5001 -t 30
```

Generate UDP Traffic

```
mininet> Server iperf -s -u -p 5002 &  
mininet> PC1 iperf -c 192.168.2.100 -u -p 5002 -b 10M -t 30
```

Kill dump

```
mininet> Server kill %tcpdump
```


Wireshark

General Traffic

After exiting mininet, open a new terminal and run: 'sudo wireshark /tmp/baseline_traffic.pcap'

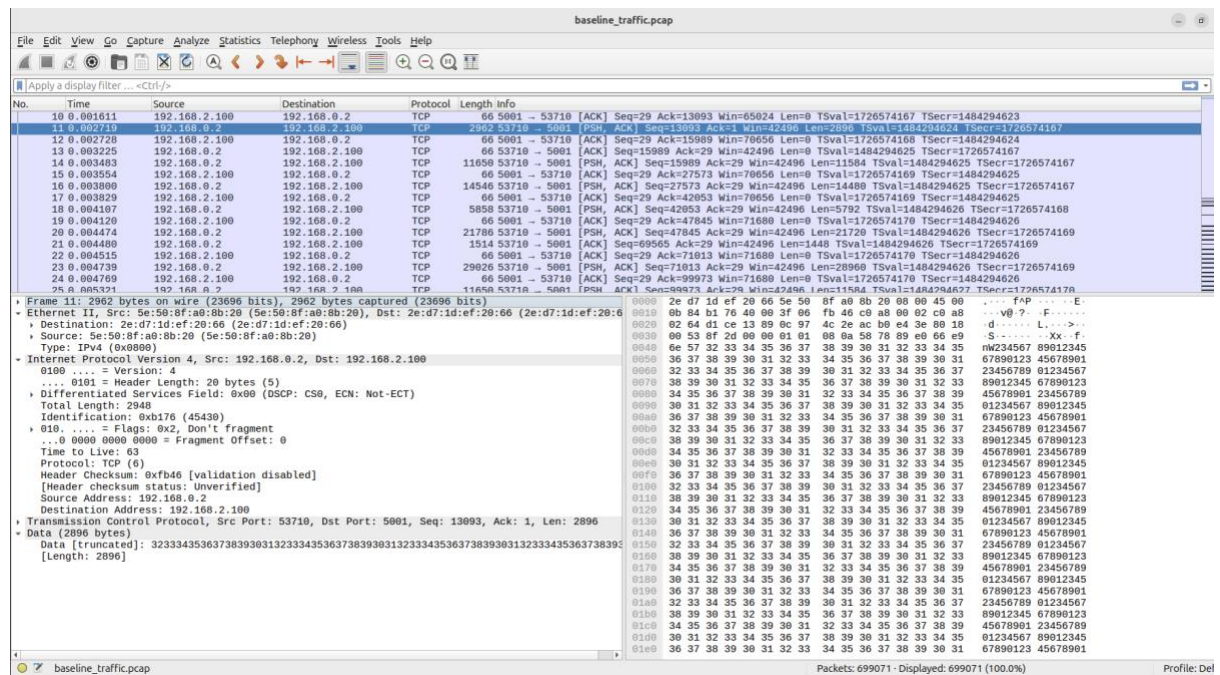


Figure 2.1.1 – Wireshark Open.

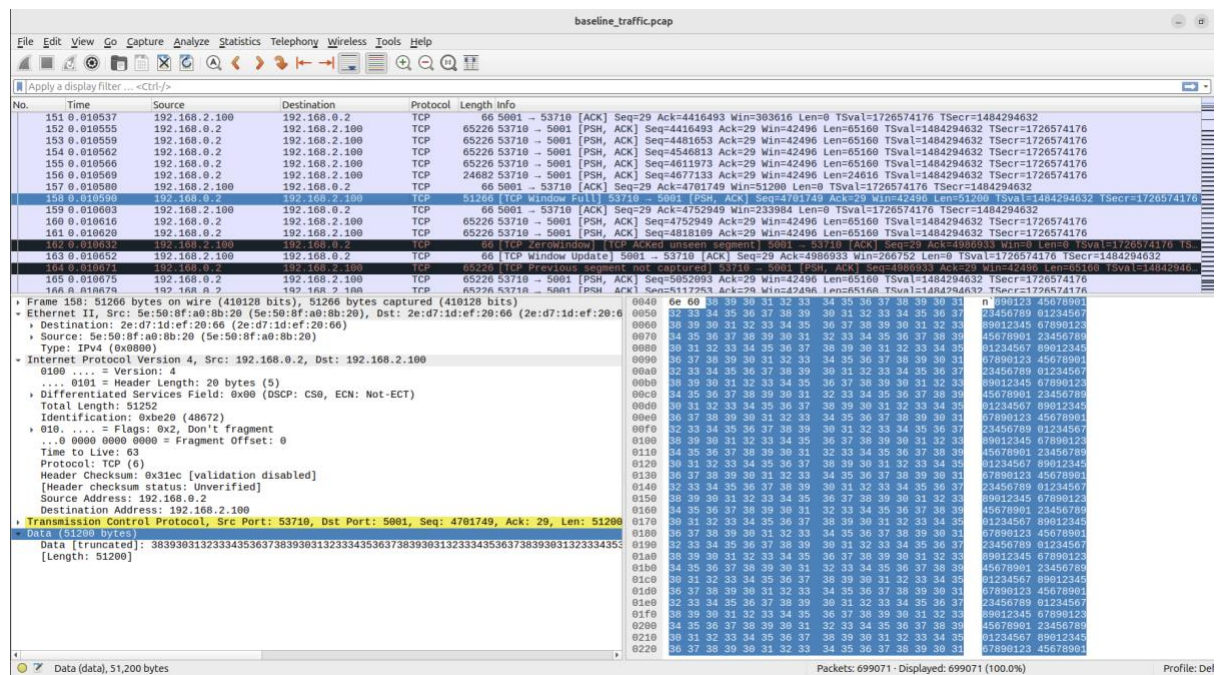


Figure 2.1.2 – Going through packets in Wireshark.

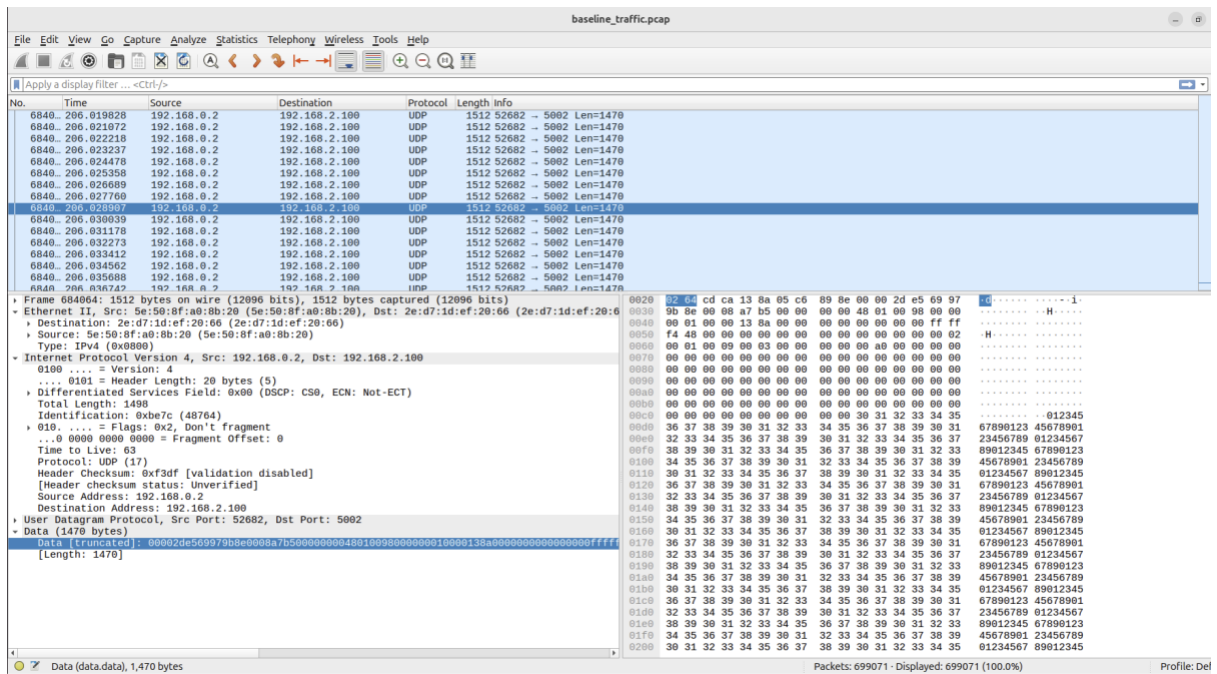


Figure 2.1.3 – Analysing different packets.

Protocol Hierarchy

Upon opening the Wireshark Protocol Hierarchy (which shows Protocol-level analysis), it showed 699,071 total frames in the baseline capture, with TCP as the dominant protocol (96.2% of packets) and a smaller UDP component (3.8% of packets), consistent with running both TCP and UDP tests.

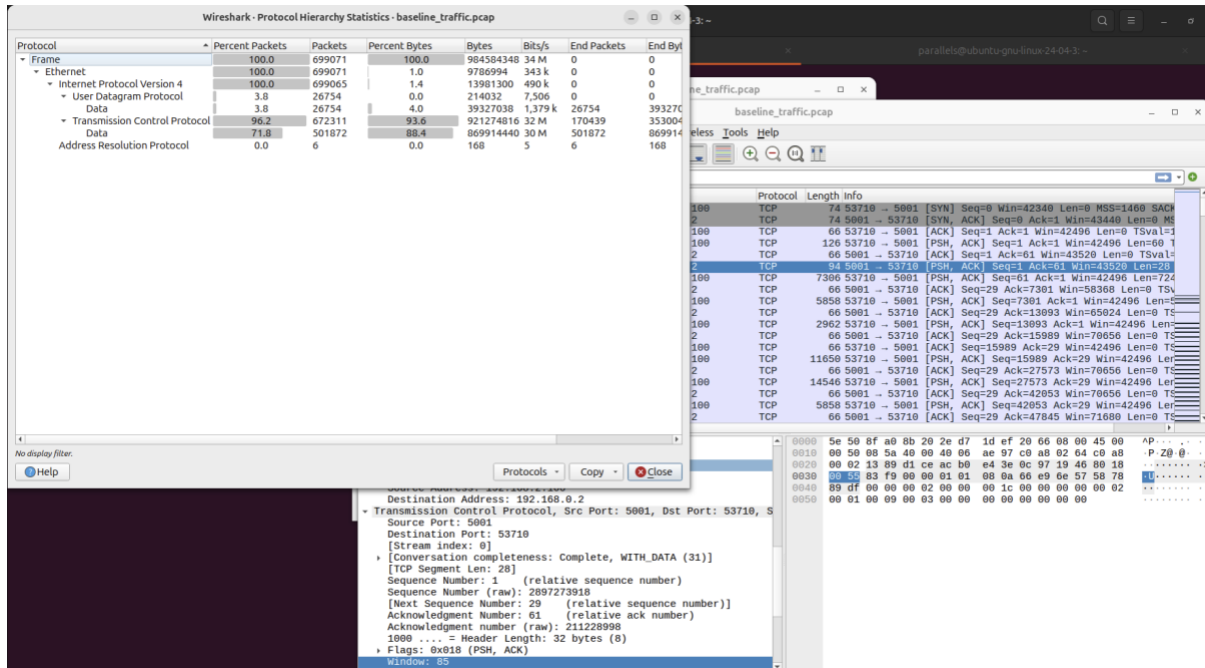


Figure 2.2.1 – Wireshark Protocol Hierarchy.

Wireshark Layer Inspection

To verify the overall traffic volume and throughput, we examined the Wireshark Conversations view filtered to all layers. As shown in Figures 2.3.1, 2.3.2, 2.3.3, and 2.3.4, the baseline capture recorded approximately 19 GB of data transferred between the two endpoints over 219 seconds, yielding a throughput of 691 Mbps in the primary direction (A → B) and 286 kbps in the return direction (B → A). This asymmetry is consistent with a bulk data transfer, such as an iPerf test.

To characterise the baseline traffic, we examined the Wireshark Conversations window across its four main tabs: Ethernet, IPv4, TCP, and UDP.

- At the Ethernet layer (Figure 2.3.1), the capture totalled 19 GB transferred between two endpoints over approximately 219 seconds at 691 Mbps, with minimal return traffic (286 kbps).
- The IPv4 tab (Figure 2.3.2) confirmed the specific hosts involved (192.168.0.2 and 192.168.2.100) and showed a slightly shorter conversation duration of 214 seconds at 708 Mbps, with the difference attributable to non-IPv4 frames (e.g., ARP) captured at the Ethernet level.
- Drilling into the transport layer, the TCP tab (Figure 2.3.3) revealed a single stream on port 5001 (the default iPerf server port) accounting for the vast majority of traffic at 19 GB across 671,907 packets.
- The UDP tab (Figure 2.3.4) showed a much smaller secondary flow on port 5002 — approximately 40 MB over 30 seconds at roughly 10 Mbps — with virtually no return traffic, consistent with a unidirectional UDP iPerf test.

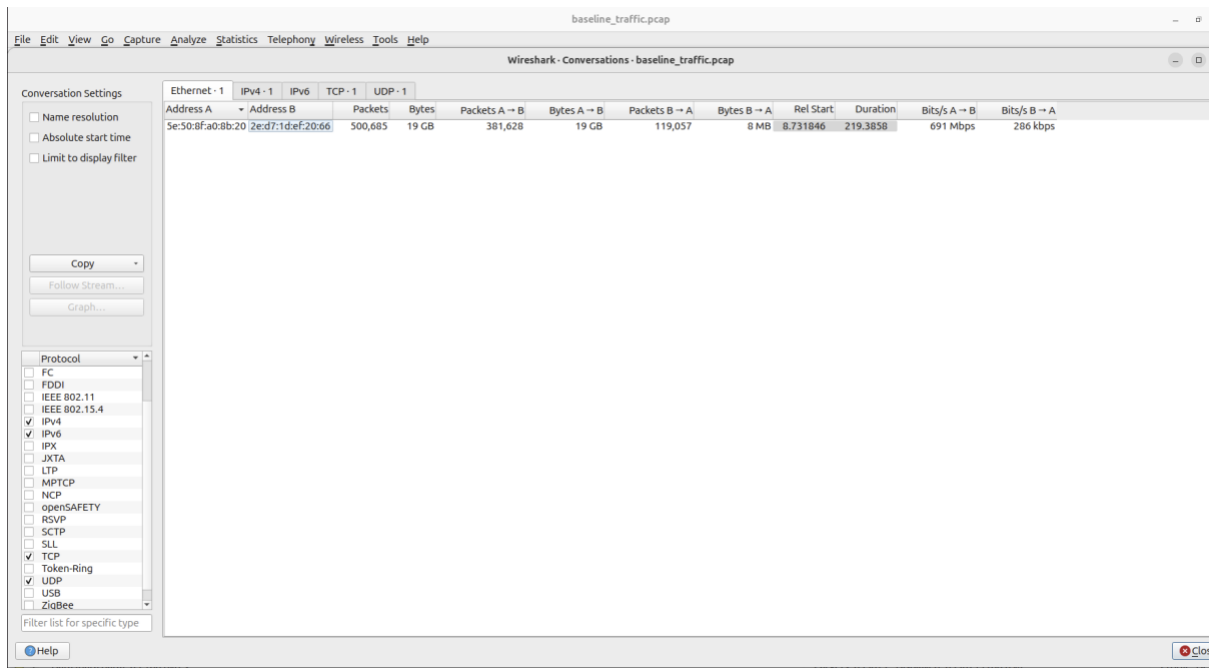


Figure 2.3.1 – Wireshark Conversations view (Ethernet tab) for the baseline capture, showing 19 GB transferred over approximately 219 seconds at an observed rate of 691 Mbps (A → B).

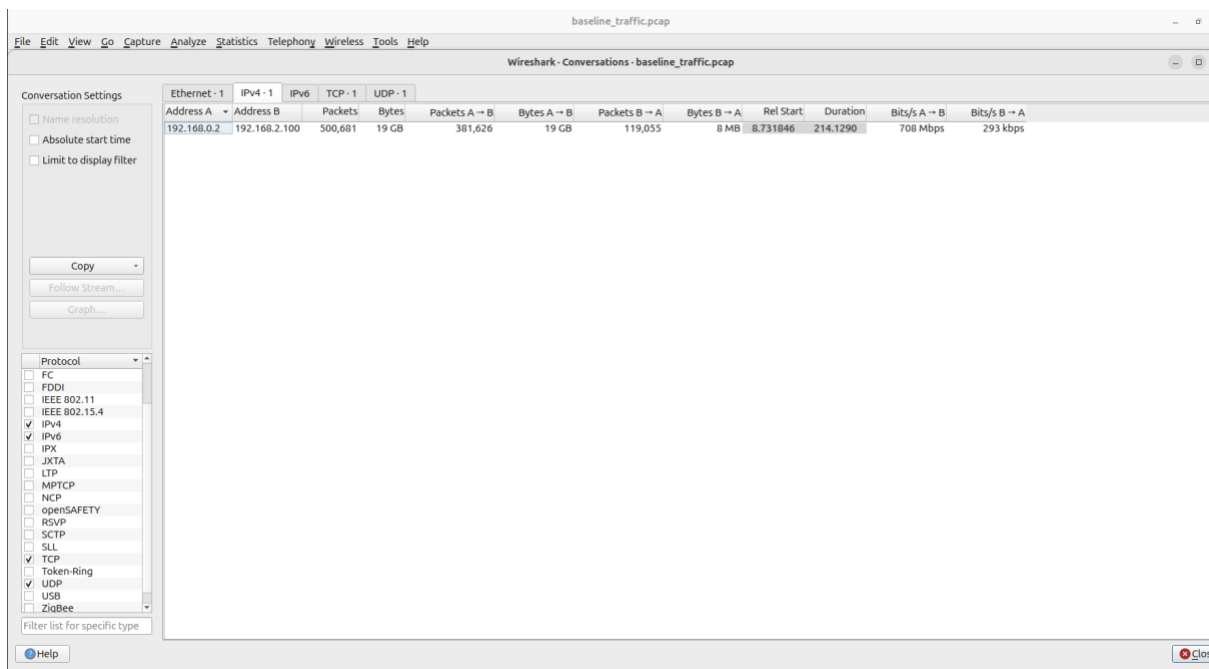


Figure 2.3.2 – Wireshark Conversations view (IPv4 tab) for the baseline capture, showing traffic between 192.168.0.2 and 192.168.2.100 with 19 GB transferred over approximately 214 seconds at 708 Mbps (A → B).

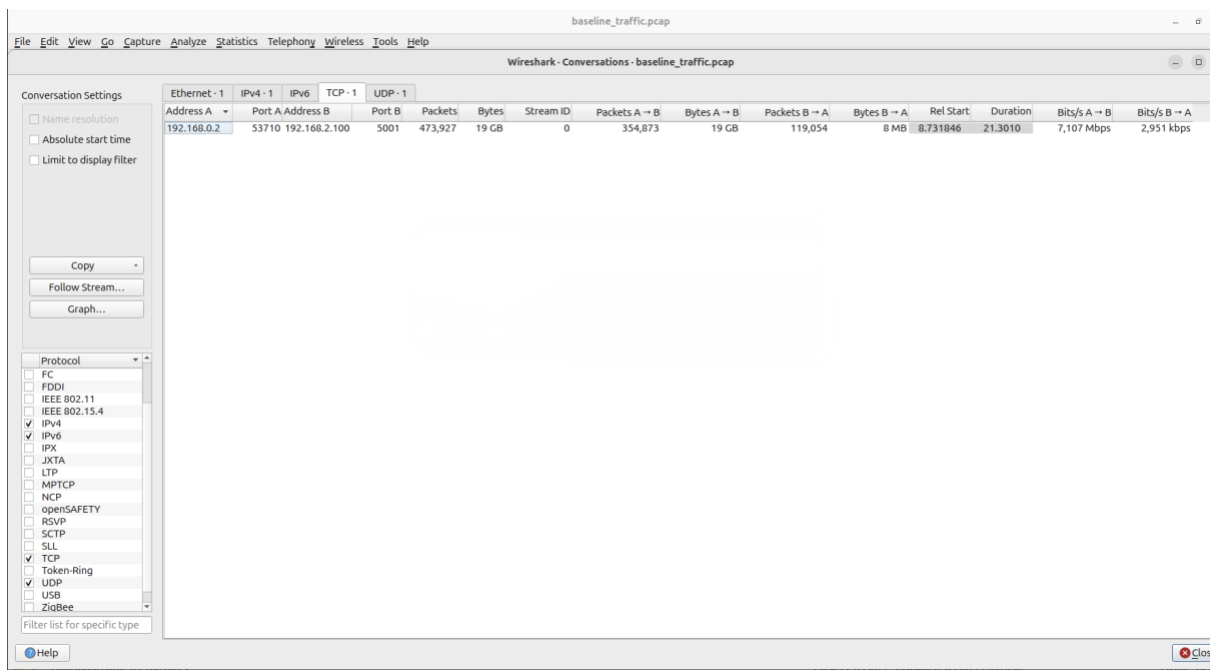


Figure 2.3.3 – Wireshark Conversations view (TCP tab) for the baseline capture, showing a single TCP stream between 192.168.0.2:54110 and 192.168.2.100:5001, with 19 GB transferred across 671,907 packets.

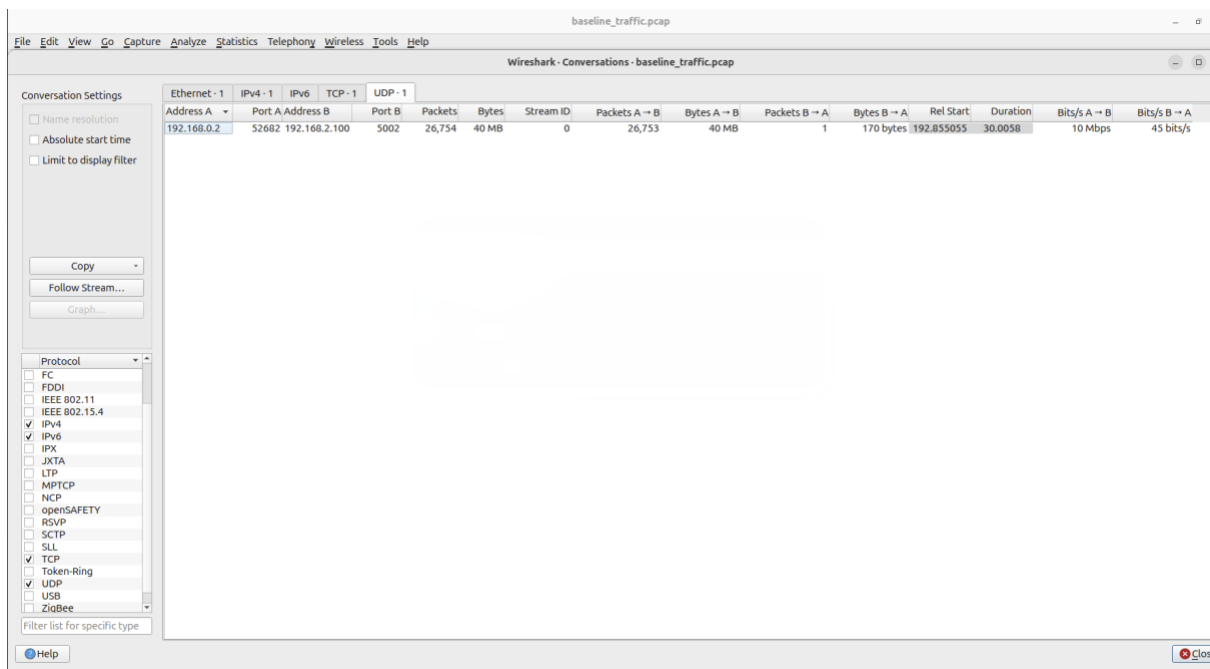


Figure 2.3.4 – Wireshark Conversations view (UDP tab) for the baseline capture, showing a single UDP flow between 192.168.0.2:52682 and 192.168.2.100:5002 with approximately 40 MB transferred over 30 seconds.

Graph Inspection

Traffic behaviour over time was then visualised using two Wireshark graphs from the baseline capture.

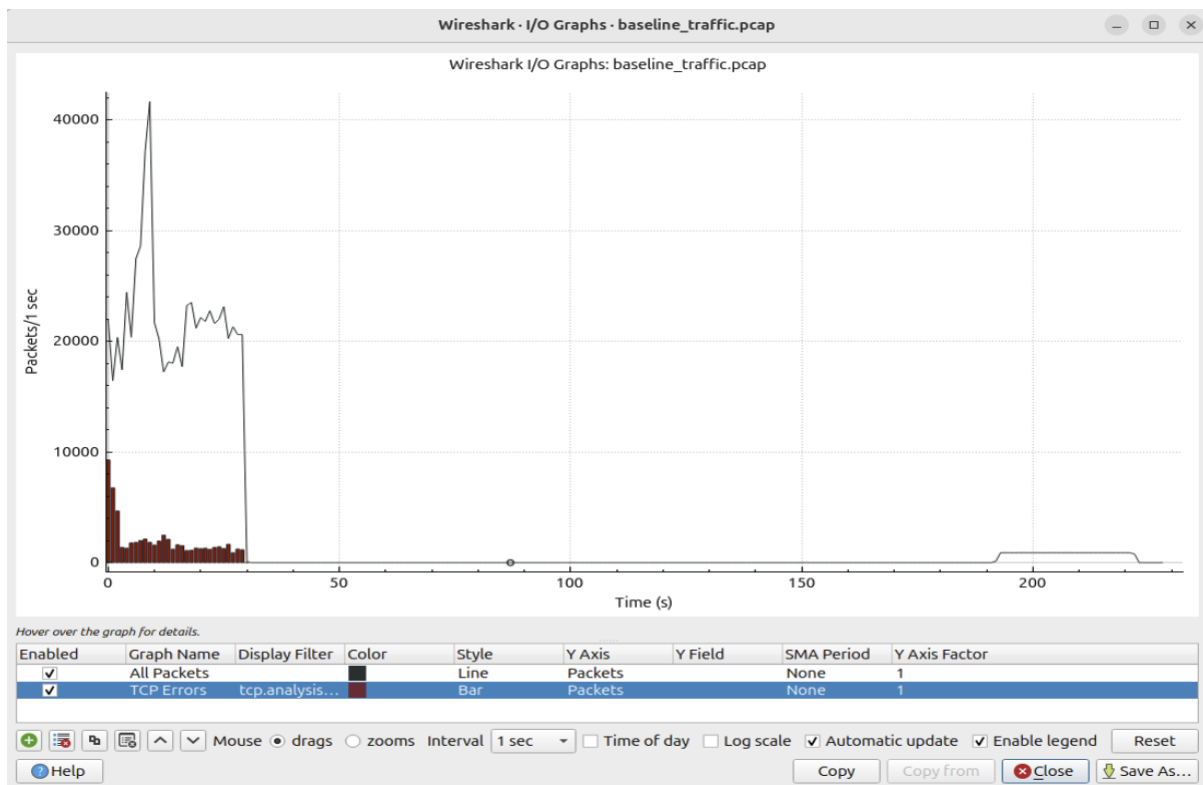


Figure 2.4.1 – I/O Graph displaying traffic volume over time.

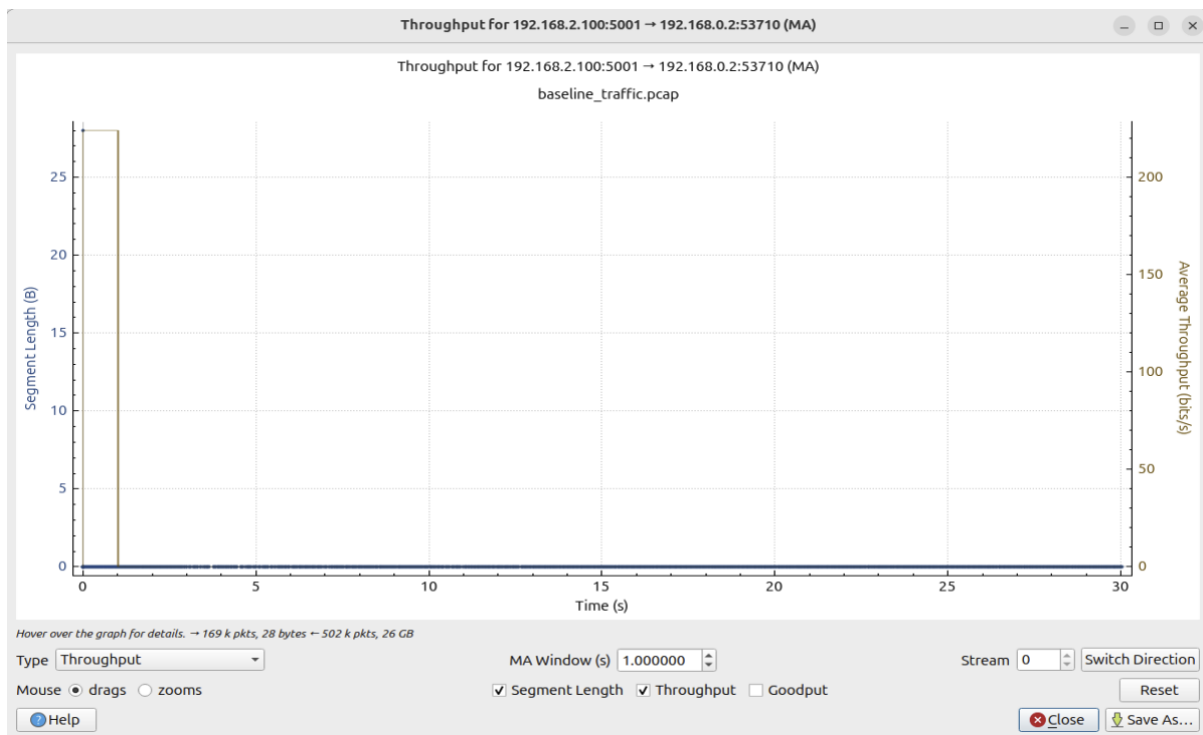


Figure 2.4.2 – TCP throughput.

Level 3 - Simulated Network Attack

Two attack classes were evaluated while normal traffic was ongoing: an ICMP echo-request flood and a TCP SYN flood. These attacks are well-known denial-of-service techniques: ICMP floods drive high packet rates, while SYN floods target TCP's connection establishment state retention.

Open 4 separate terminal windows using xterm: *mininet> xterm Server PC1 PC2 PC3*

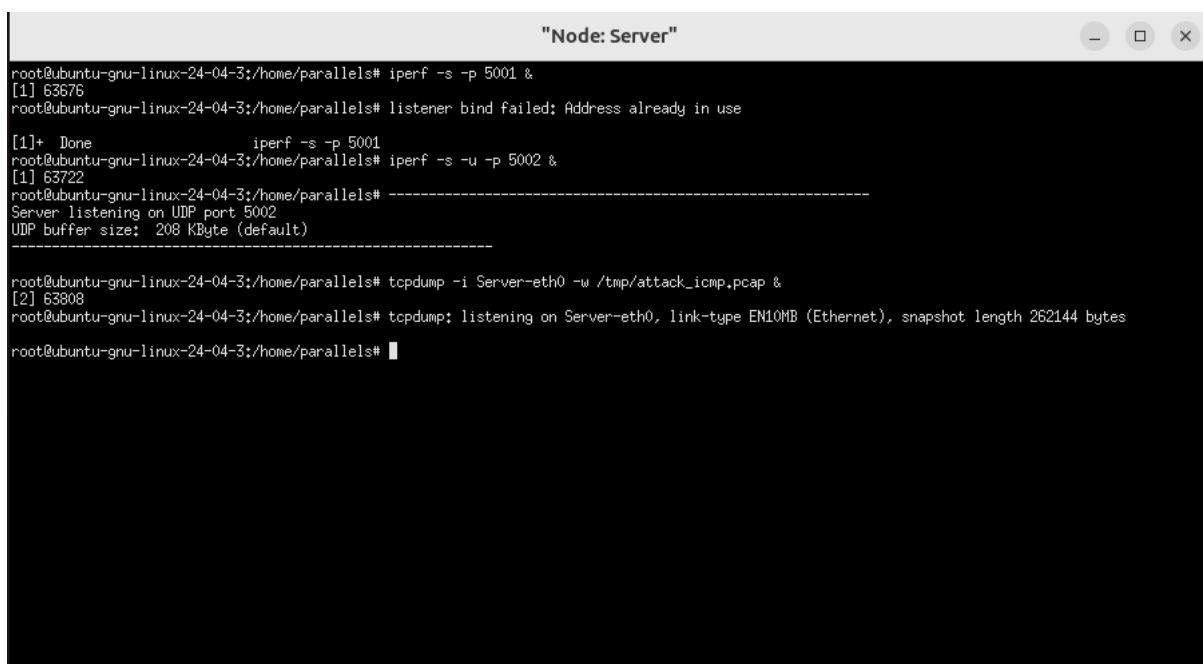
Server-Side Setup for ICMP Flood Attack Capture

In server:

```
Server> iperf -s -p 5001 &
```

```
Server> iperf -s -u -p 5002 &
```

```
Server> tcpdump -i Server-eth0 -w /tmp/attack_icmp.pcap &
```



```
"Node: Server"
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -s -p 5001 &
[1] 63676
root@ubuntu-gnu-linux-24-04-3:/home/parallels# listener bind failed: Address already in use
[1]+  Done                  iperf -s -p 5001
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -s -u -p 5002 &
[1] 63722
root@ubuntu-gnu-linux-24-04-3:/home/parallels# -----
Server listening on UDP port 5002
UDP buffer size:  208 KByte (default)
-----
root@ubuntu-gnu-linux-24-04-3:/home/parallels# tcpdump -i Server-eth0 -w /tmp/attack_icmp.pcap &
[2] 63808
root@ubuntu-gnu-linux-24-04-3:/home/parallels# tcpdump: listening on Server-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
root@ubuntu-gnu-linux-24-04-3:/home/parallels#
```

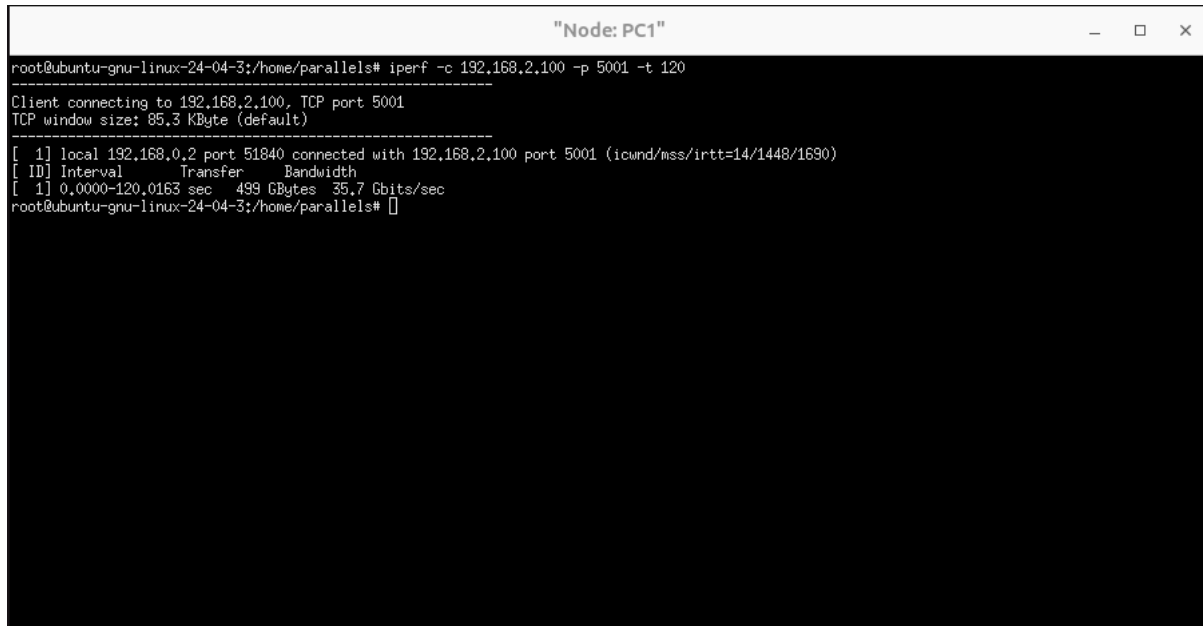
Figure 3.1.1 – The Server (192.168.2.100) was prepared by starting iPerf listeners on both TCP port 5001 and UDP port 5002, followed by a tcpdump capture on the server-facing interface.

Attack 1: ICMP Flood (Ping Flood)

Generate Baseline TCP Traffic

In PC1:

```
PC1> iperf -c 192.168.2.100 -p 5001 -t 120
```



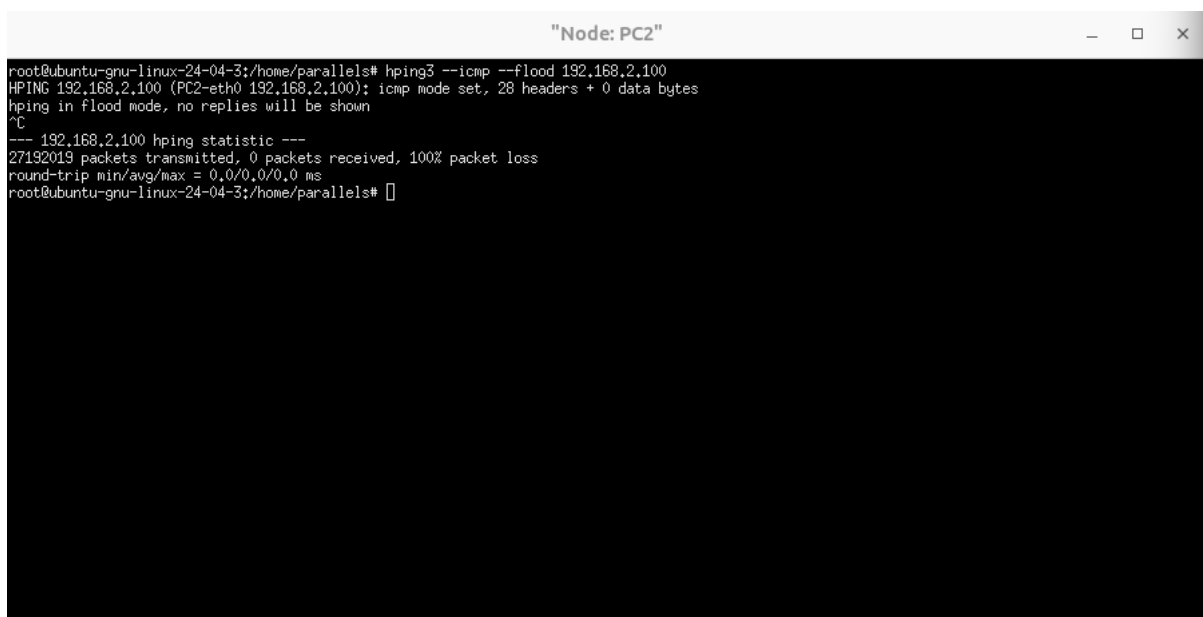
```
"Node: PC1"
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -c 192.168.2.100 -p 5001 -t 120
-----
Client connecting to 192.168.2.100, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[  1] local 192.168.0.2 port 51840 connected with 192.168.2.100 port 5001 (icwnd/mss/irrt=14/1448/1690)
[ 10] Interval      Transfer      Bandwidth
[  1] 0.0000-120.0163 sec  499 GBytes  35.7 Gbits/sec
root@ubuntu-gnu-linux-24-04-3:/home/parallels#
```

Figure 3.2.1 – A TCP throughput test was initiated from PC1 (192.168.0.2) to the iPerf server (192.168.2.100) on port 5001 with a duration of 120 seconds.

Launch ICMP flood

In PC2:

```
PC2> hping3 --icmp --flood 192.168.2.100
```



```
"Node: PC2"
root@ubuntu-gnu-linux-24-04-3:/home/parallels# hping3 --icmp --flood 192.168.2.100
HPING 192.168.2.100 (PC2-eth0 192.168.2.100): icmp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 192.168.2.100 hping statistic ---
27192019 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
root@ubuntu-gnu-linux-24-04-3:/home/parallels#
```

Figure 3.2.2 – ICMP flood attack initiated.

Run for ~30 seconds, then press 'CTRL + C' in the PC2 xterm to stop

Inspect Traffic:

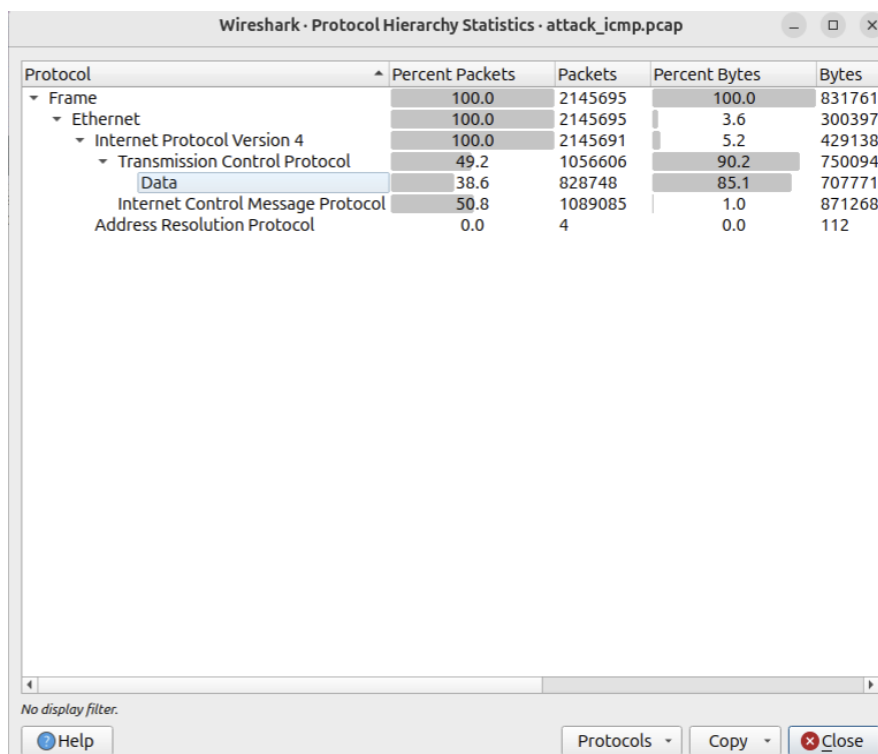


Figure 3.3.1 – Wireshark Protocol Hierarchy for the ICMP flood capture, showing approximately 2,145,695 total packets split nearly evenly between TCP (49.2%) and ICMP (50.8%).

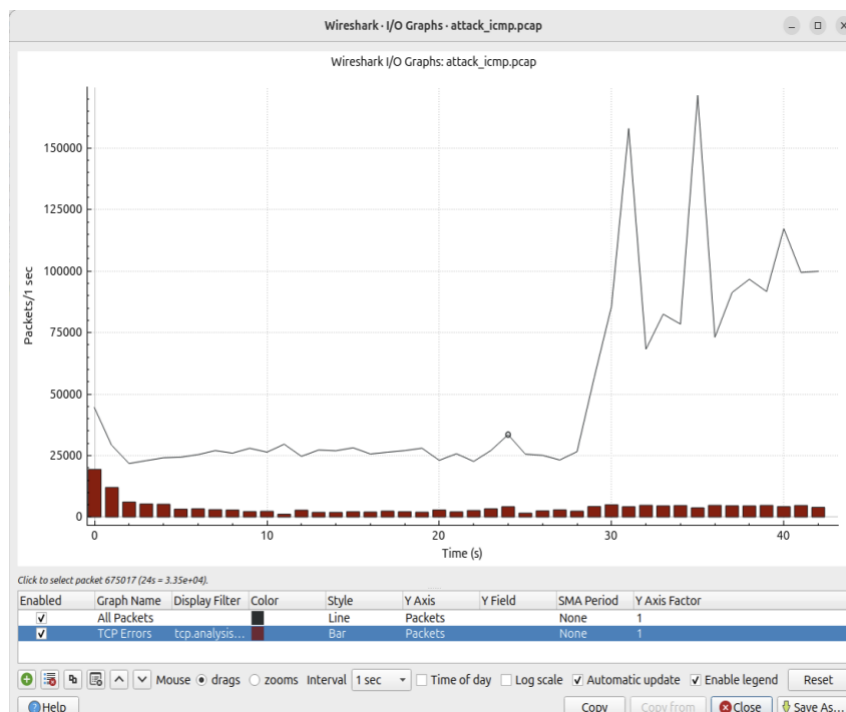


Figure 3.3.2 — Wireshark I/O Graph for the ICMP flood capture, illustrating a sharp burst of TCP errors (red bars) in the first few seconds followed by sustained but irregular packet activity, with notable spikes around the 25- and 30-second marks.

Wireshark - Conversations - attack_icmp.pcap												
Ethernet: 1	IPv4: 1	IPv6	TCP: 1	UDP								
Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A	
42:a2:40:f9:0b:43	6a:fc:f8:06:20:01	179,651	8 GB	138,571	8 GB	41,080	3 MB	2.151554	7.2222	8,815 Mbps	3,003 kbps	

Figure 3.3.3 – Ethernet tab of Wireshark Conversations view during the ICMP flood capture.

Wireshark - Conversations - attack_icmp.pcap												
Ethernet: 1	IPv4: 1	IPv6	TCP: 1	UDP								
Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A	
192.168.0.2	192.168.2.100	332,489	14 GB	253,378	14 GB	79,111	5 MB	2.151554	12.8983	8,834 Mbps	3,238 kbps	

Figure 3.3.4 – IPv4 tab of Wireshark Conversations view during the ICMP flood capture.

Wireshark - Conversations - attack_icmp.pcap												
Ethernet: 1	IPv4: 1	IPv6	TCP: 1	UDP								
Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A	

Figure 3.3.5 – IPv6 tab of Wireshark Conversations view during the ICMP flood capture, showing no IPv6 traffic.

Wireshark - Conversations - attack_icmp.pcap														
Ethernet: 1		IPv4: 1	IPv6	TCP: 1	UDP									
Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
192.168.0.2	36276	192.168.2.100	5001	593,397	26 GB	0	454,908	26 GB	138,489	9 MB	2.151554	22.6662	9,020 Mbps	3,228 kbps

Figure 3.3.6 – TCP tab of Wireshark Conversations view during the ICMP flood capture.

Wireshark - Conversations - attack_icmp.pcap													
Ethernet: 1	IPv4: 1	IPv6	TCP: 1	UDP									
Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A

Figure 3.3.7 – UDP tab of Wireshark Conversations view during the ICMP flood capture, showing no UDP traffic.

Attack 2: TCP SYN Flood

Launch SYN flood

In Server

```
Server> tcpdump -U -i Server-eth0 -w /tmp/attack_syn.pcap &
```

```
root@ubuntu-gnu-linux-24-04-3:/home/parallels# tcpdump -i Server-eth0 -w /tmp/attack_syn.pcap &
[4] 6277
root@ubuntu-gnu-linux-24-04-3:/home/parallels# tcpdump: listening on Server-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
[ 2] local 192.168.2.100 port 5001 connected with 192.168.0.2 port 34564 (icwnd/mss/irtt=14/1448/1526)
[ ID] Interval      Transfer      Bandwidth
[ 2] 0.0000-120.0030 sec  366 GBytes  26.2 Gbits/sec
```

Figure 3.4.1 – Launch SYN Flood attack.

Start normal traffic

```
PC1> iperf -c 192.168.2.100 -p 5001 -t 120
```

```
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -c 192.168.2.100 -p 5001 -t 120
-----
Client connecting to 192.168.2.100, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 192.168.0.2 port 34564 connected with 192.168.2.100 port 5001 (icwnd/mss/irtt=14/1448/1619)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-120.0134 sec  366 GBytes  26.2 Gbits/sec
```

Figure 3.4.2 – iperf TCP throughput test from PC1 to server at 192.168.2.100 on port 5001, run over 120 seconds.

Launch SYN flood from PC3 targeting Server port 5001

```
PC3> hping3 -S --flood -p 5001 192.168.2.100
```

```
root@ubuntu-gnu-linux-24-04-3:/home/parallels# hping3 -S --flood -p 5001 192.168.2.100
HPING 192.168.2.100 (PC3-eth0 192.168.2.100): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 192.168.2.100 hping statistic ---
4521254 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Figure 3.4.3 – SYN flood attack launched from PC3 against the server (192.168.2.100) on port 5001 using hping3 in flood mode.

Run for ~30 seconds, then press 'CTRL + C' in the PC3 xterm to stop

SYN Flood with IP Spoofing

PC3> PC3 hping3 -S --flood -p 5001 --rand-source 192.168.2.100 &

```
root@ubuntu-gnu-linux-24-04-3:/home/parallels# hping3 -S --flood -p 5001 --rand-source 192.168.2.100
HPING 192.168.2.100 (PC3-eth0 192.168.2.100): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 192.168.2.100 hping statistic ---
7041972 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Figure 3.4.4 – SYN flood attack with IP spoofing (--rand-source) launched from PC3 against 192.168.2.100 on port 5001 using hping3.

Run for ~30 seconds, then press 'CTRL + C' in the PC3 xterm to stop

Inspect in Wireshark

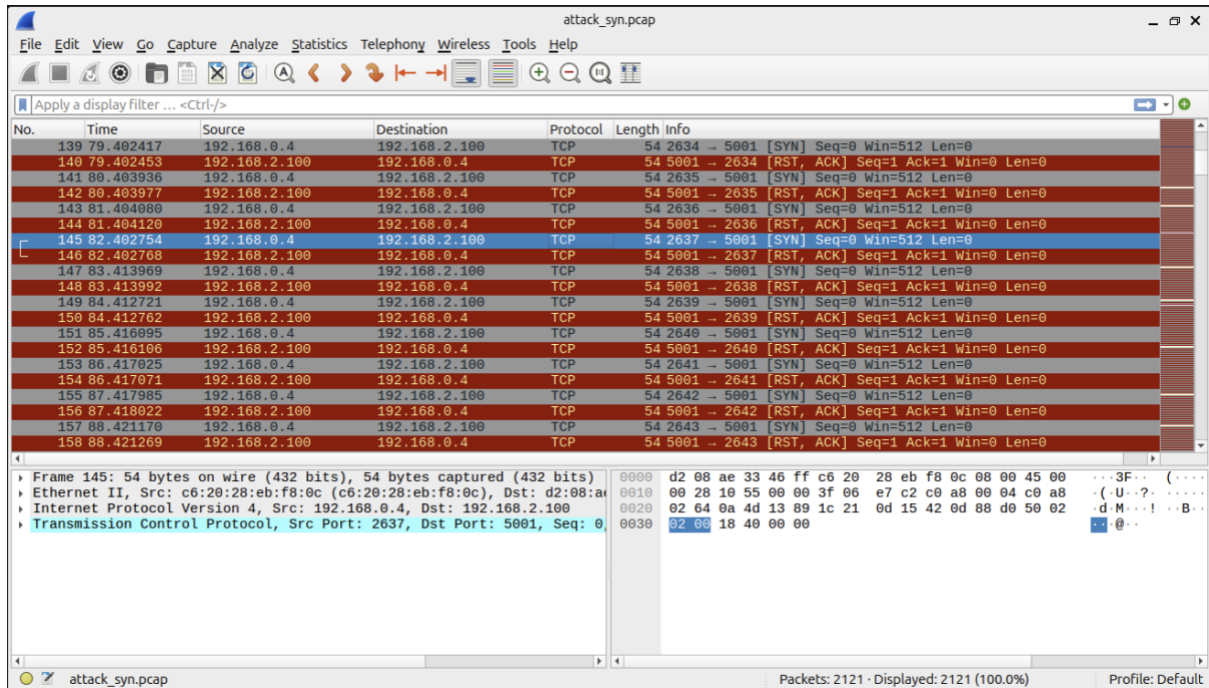


Figure 3.5.1 – Wireshark capture of the SYN flood attack (attack_syn.pcap), showing the rapid alternation of SYN packets from 192.168.0.4 to 192.168.2.100 on port 5001 and corresponding RST/ACK replies.

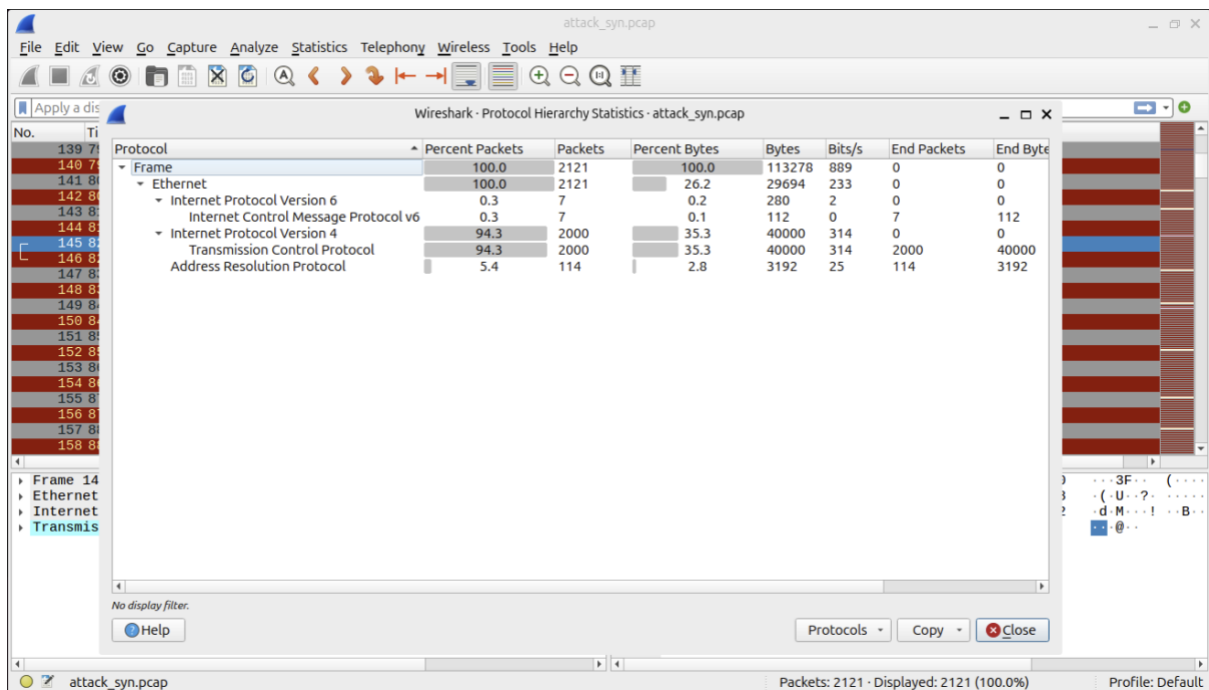


Figure 3.5.2 – Wireshark Protocol Hierarchy Statistics for attack_syn.pcap.

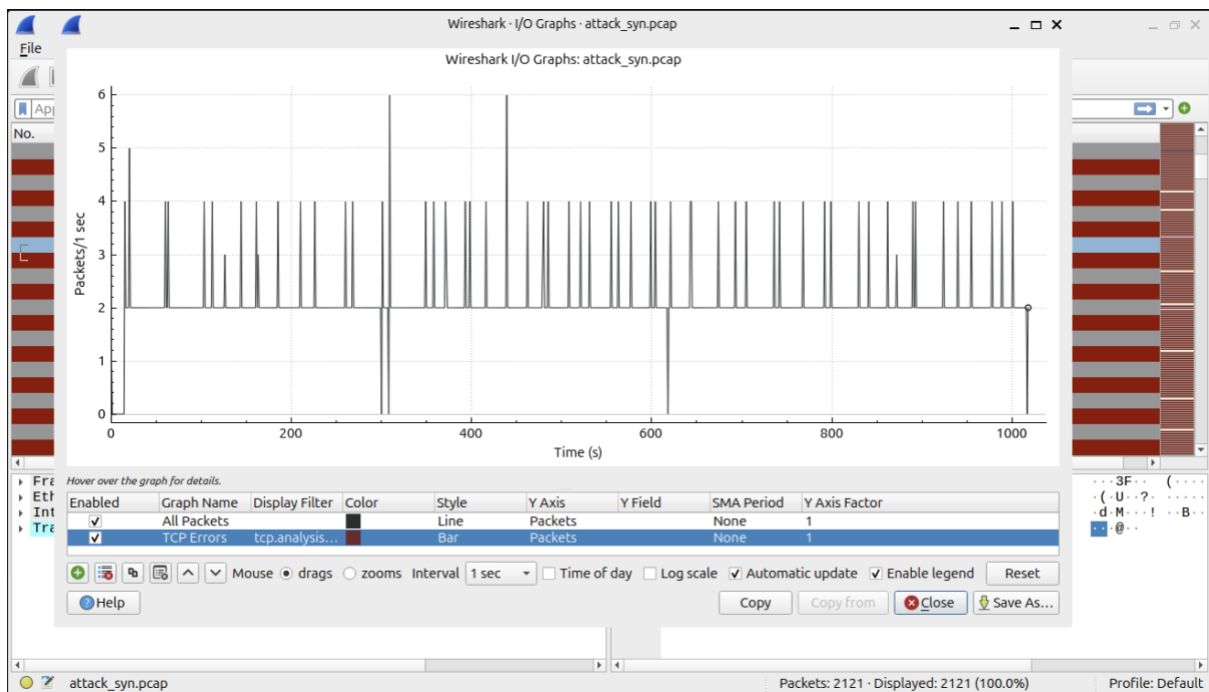


Figure 3.5.3 – Wireshark I/O Graph for `attack_syn.pcap`, plotting all packets and TCP errors over ~1,000 seconds.

The figure shows the Wireshark Conversations view for the file `attack_syn.pcap` at the Ethernet layer. The table displays 4 conversations between various MAC addresses.

Address A		Address B		Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Durati
c6:20:28:eb:f8:0c	33:33:00:00:00:02			4	280 bytes	4	280 bytes	0	0 bytes	0.000000	872.448
c6:20:28:eb:f8:0c	ff:ff:ff:ff:ff:ff			1	42 bytes	1	42 bytes	0	0 bytes	15.183061	0.0000
d2:08:ae:33:46:ff	33:33:00:00:00:02			3	210 bytes	3	210 bytes	0	0 bytes	20.480301	458.7517
d2:08:ae:33:46:ff	c6:20:28:eb:f8:0c			2,113	113 kB	1,057	56 kB	1,056	56 kB	15.183082	1003.187

Figure 3.5.4 – Wireshark Conversations view for `attack_syn.pcap` at the Ethernet layer, showing 4 conversations.

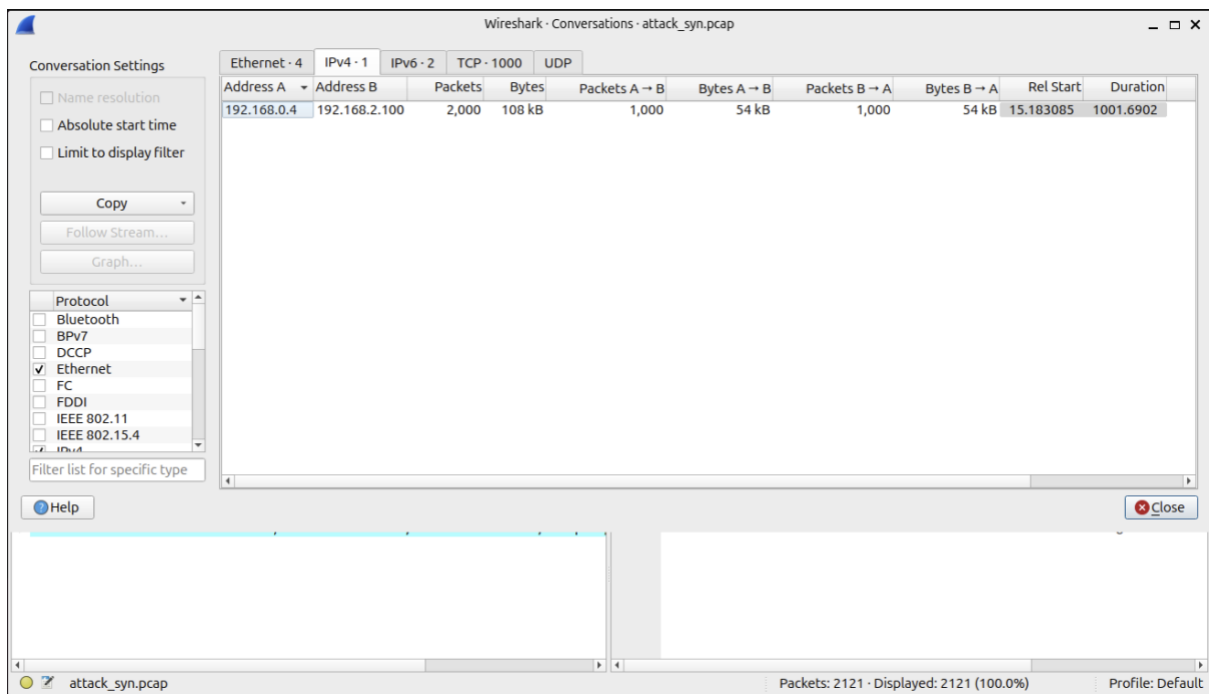


Figure 3.5.5 – Wireshark Conversations view (IPv4 tab) for attack_syn.pcap, showing a single conversation between 192.168.0.4 and 192.168.2.100.

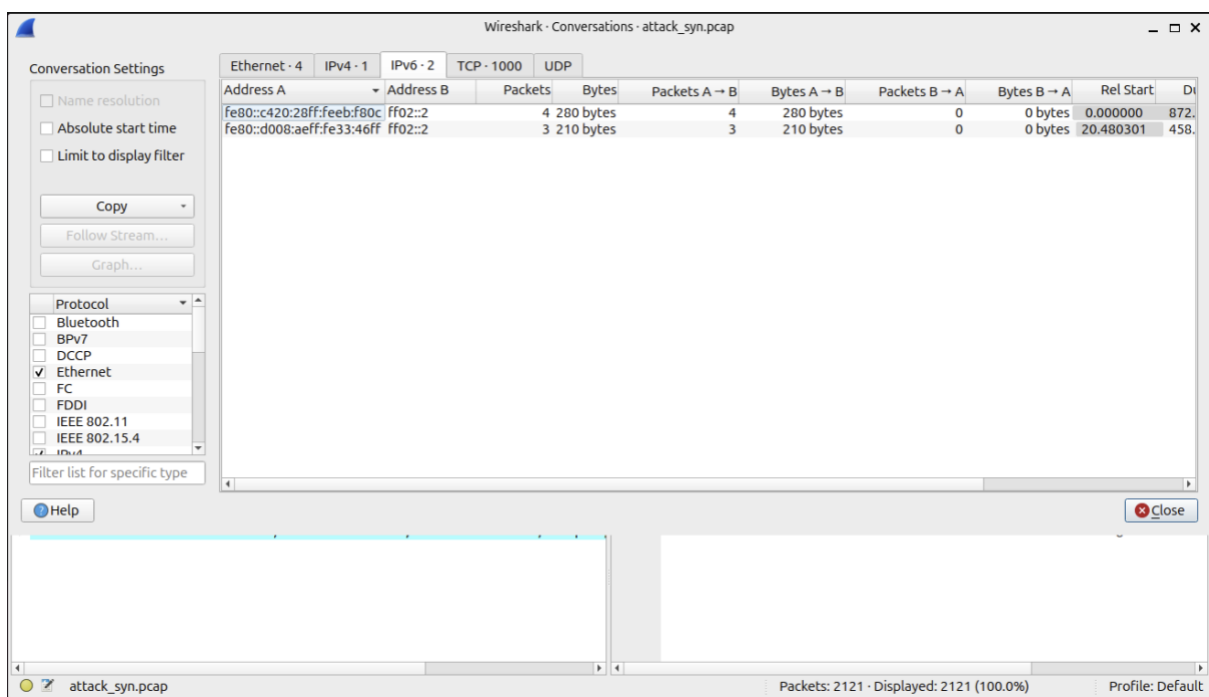


Figure 3.5.6 – Wireshark Conversations view (IPv6 tab) for attack_syn.pcap, showing two link-local IPv6 conversations.

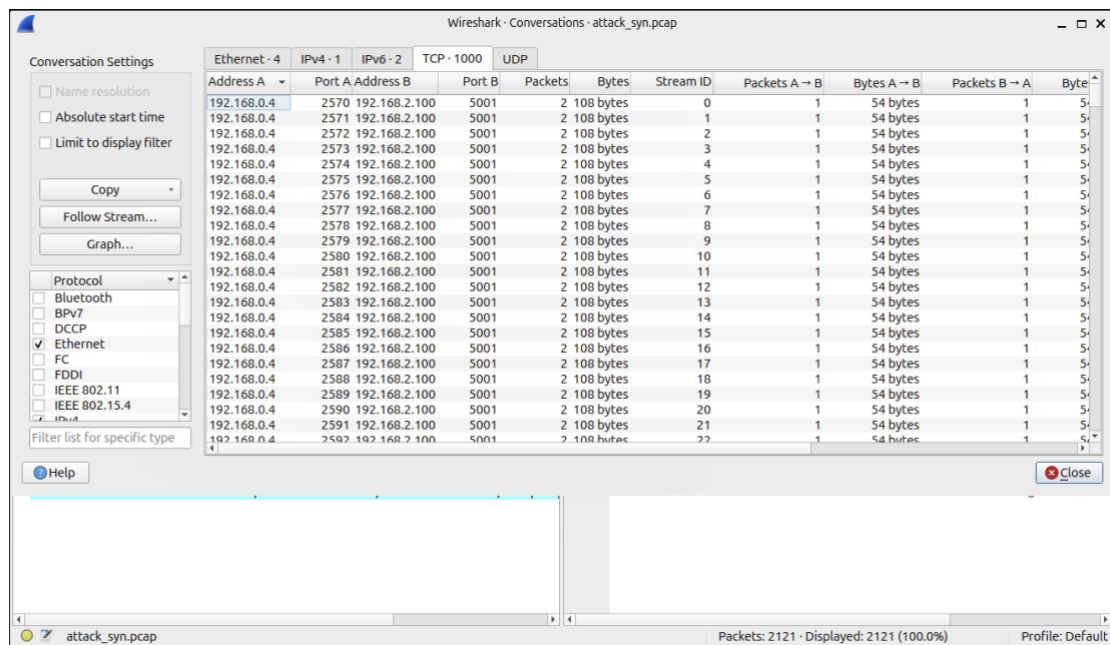


Figure 3.5.7 – Figure 3.5.7 – Wireshark Conversations view (TCP tab) for attack_syn.pcap, showing numerous individual TCP streams from 192.168.0.4 to 192.168.2.100 on port 5001.

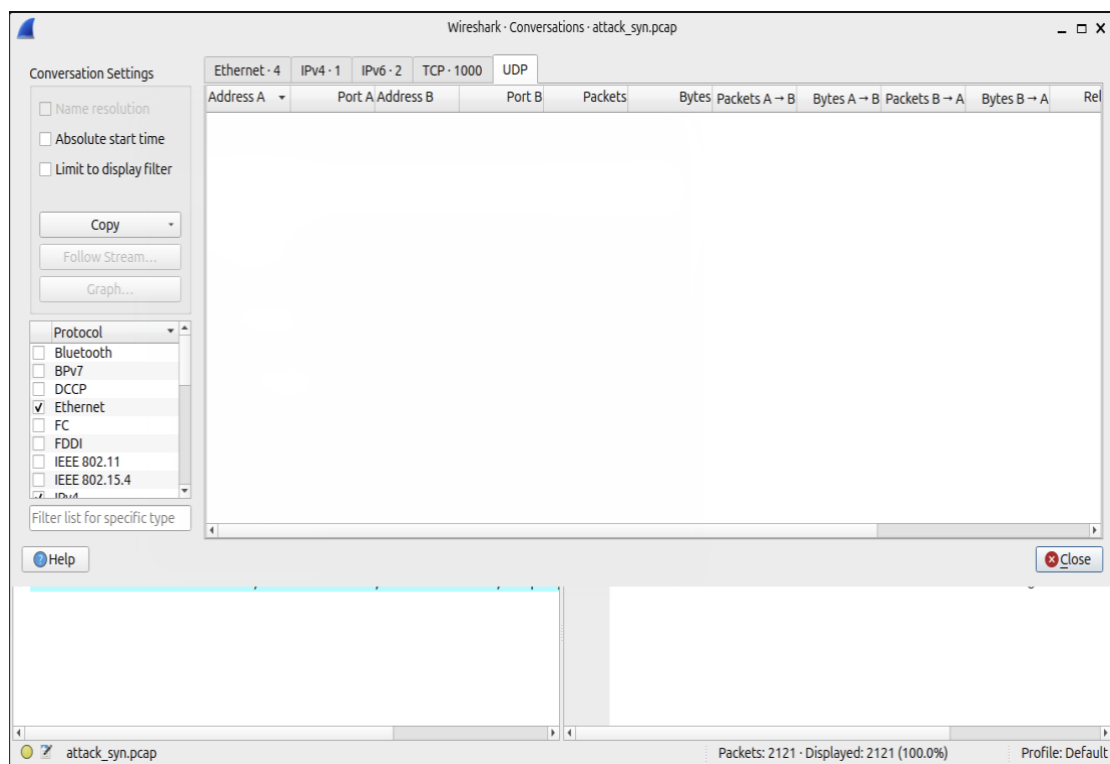


Figure 3.5.8 – Wireshark Conversations view (UDP tab) for attack_syn.pcap, showing no UDP conversations, confirming the attack traffic was entirely TCP-based with no UDP component present in the capture.

Level 4 - Network Defence with Firewall

Router-based firewall rules were deployed using iptables/Netfilter, which provides packet filtering and related capabilities for Linux systems.

Defend ICMP

Enable Firewall to block ICMP Requests

Firewall on the Router

```
mininet> Router1 iptables -A FORWARD -p icmp --icmp-type echo-request -m limit --limit 1/s -  
-limit-burst 4 -j ACCEPT
```

```
mininet> Router1 iptables -A FORWARD -p icmp --icmp-type echo-request -j DROP
```

For ICMP flooding, the defence used rate limiting on ICMP echo requests (allow a small burst, then drop further echo requests). iptables documentation defines rate limiting semantics via --limit and --limit-burst.

Verify the firewall rules

```
mininet> Router1 iptables -L -n -v
```

```
mininet> Router1 iptables -L -n -v  
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)  
pkts bytes target      prot opt in      out     source      destination  
  
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)  
pkts bytes target      prot opt in      out     source      destination  
  0    0 ACCEPT      1    -- *      *       0.0.0.0/0    0.0.0.0/0    icmp type 8 limit: avg 1/sec burst 4  
  0    0 DROP       1    -- *      *       0.0.0.0/0    0.0.0.0/0    icmp type 8  
  
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)  
pkts bytes target      prot opt in      out     source      destination  
mininet>
```

Figure 4.1.1 – Verification that the firewall rules has been successfully registered by the router.

Perform ICMP Attack

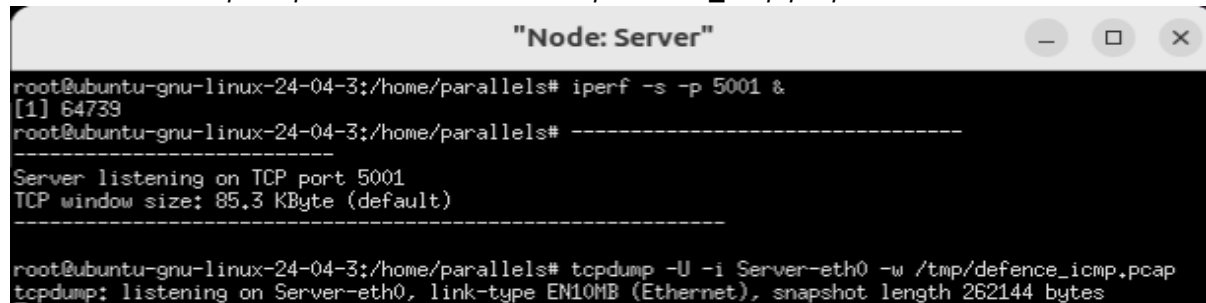
Open Xterm

```
mininet> xterm Server PC1 PC2 PC3
```

Server

```
Server> iperf -s -p 5001 &
```

```
Server> tcpdump -U -i Server-eth0 -w /tmp/defence_icmp.pcap
```

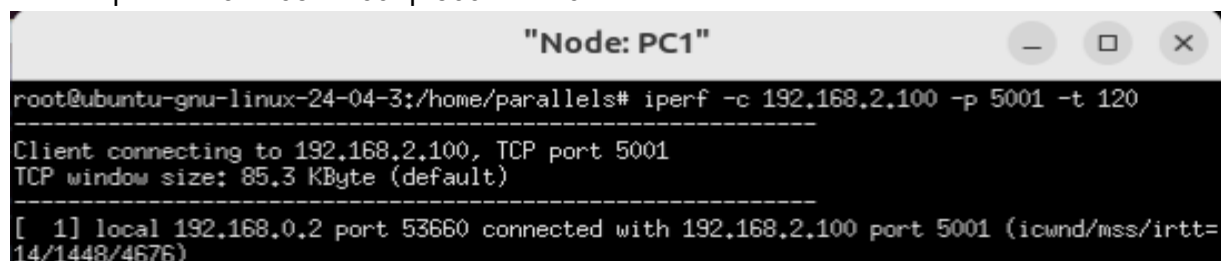


```
"Node: Server"
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -s -p 5001 &
[1] 64739
root@ubuntu-gnu-linux-24-04-3:/home/parallels# -----
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----
root@ubuntu-gnu-linux-24-04-3:/home/parallels# tcpdump -U -i Server-eth0 -w /tmp/defence_icmp.pcap
tcpdump: listening on Server-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

Figure 4.1.2 – Server node initialisation for the ICMP attack scenario. *iperf* is started as a background TCP server on port 500, and *tcpdump* begins capturing traffic on *Server-eth0* to */tmp/defence_icmp.pcap*.

PC1

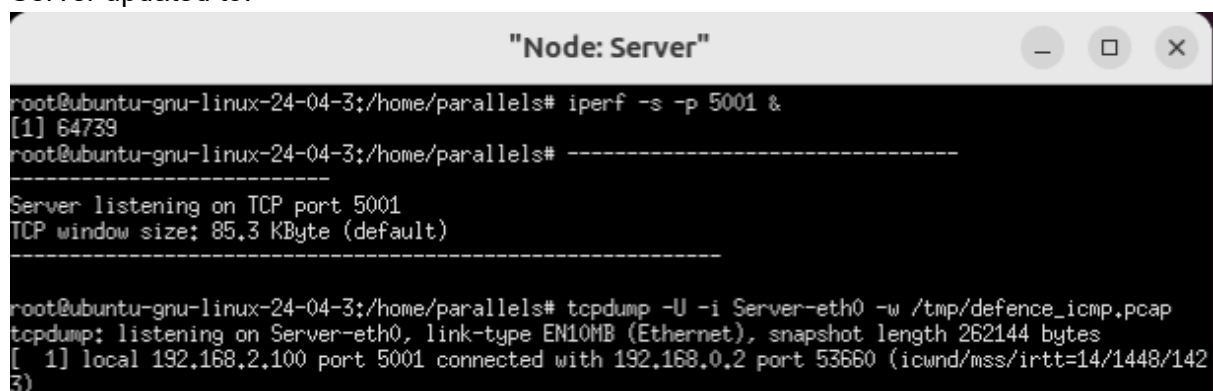
```
iperf -c 192.168.2.100 -p 5001 -t 120
```



```
"Node: PC1"
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -c 192.168.2.100 -p 5001 -t 120
-----
Client connecting to 192.168.2.100, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 192.168.0.2 port 53660 connected with 192.168.2.100 port 5001 (icwnd/mss/irrt=14/1448/4676)
```

Figure 4.1.3 – PC1 initiates an *iperf* TCP client connection to the server (192.168.2.100) on port 5001 for 120 seconds.

Server updated to:

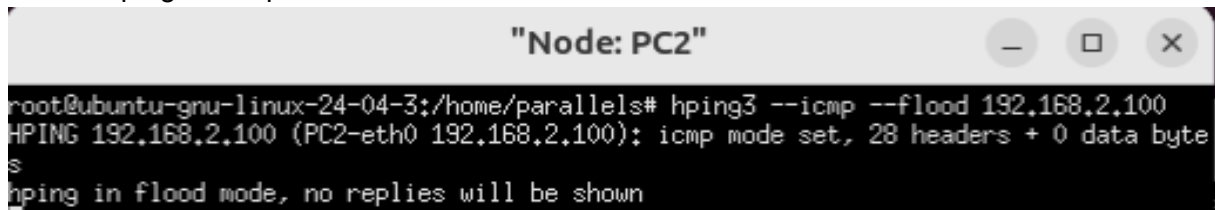


```
"Node: Server"
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -s -p 5001 &
[1] 64739
root@ubuntu-gnu-linux-24-04-3:/home/parallels# -----
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----
root@ubuntu-gnu-linux-24-04-3:/home/parallels# tcpdump -U -i Server-eth0 -w /tmp/defence_icmp.pcap
tcpdump: listening on Server-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
[ 1] local 192.168.2.100 port 5001 connected with 192.168.0.2 port 53660 (icwnd/mss/irrt=14/1448/1423)
```

Figure 4.1.4 – Server node updated view confirming the *iperf* connection from PC1 (192.168.0.2:53660) has been accepted, with *tcpdump* actively capturing on *Server-eth0*.

PC2

```
hping3 --icmp --flood 192.168.2.100
```

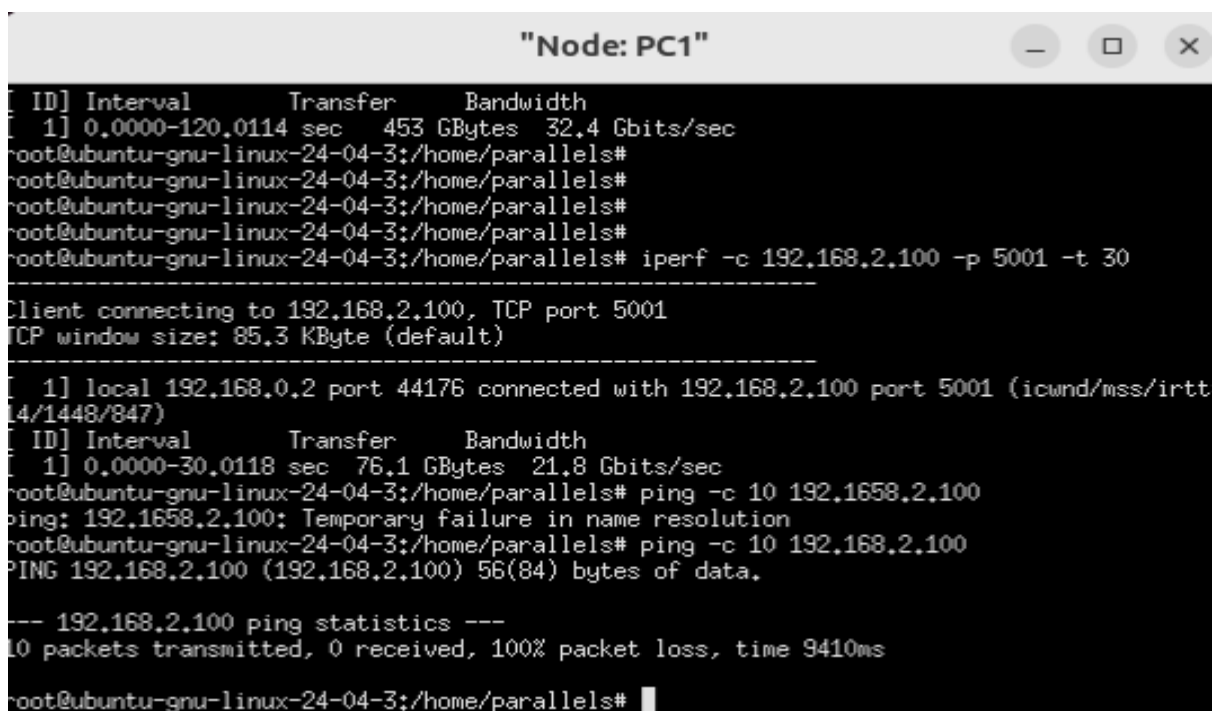
A terminal window titled "Node: PC2" showing the execution of the hping3 command. The prompt is root@ubuntu-gnu-linux-24-04-3:/home/parallels#. The command entered is hping3 --icmp --flood 192.168.2.100. The output shows "HPING 192.168.2.100 (PC2-eth0 192.168.2.100): icmp mode set, 28 headers + 0 data bytes" and "hping in flood mode, no replies will be shown".

```
"Node: PC2"
root@ubuntu-gnu-linux-24-04-3:/home/parallels# hping3 --icmp --flood 192.168.2.100
HPING 192.168.2.100 (PC2-eth0 192.168.2.100): icmp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

Figure 4.1.5 – PC2 launches an ICMP flood attack against the server (192.168.2.100) using hping3 in flood mode.

PC1

```
iperf -c 192.168.2.100 -p 5001 -t 30
ping -c 10 192.168.2.100
```

A terminal window titled "Node: PC1" showing the results of iperf and ping tests. The iperf test shows a bandwidth of 32.4 Gbits/sec. The ping test shows a temporary failure in name resolution and a 100% packet loss. The prompt is root@ubuntu-gnu-linux-24-04-3:/home/parallels#.

```
"Node: PC1"
[ ID] Interval      Transfer      Bandwidth
[  1] 0.0000-120.0114 sec  453 GBytes  32.4 Gbits/sec
root@ubuntu-gnu-linux-24-04-3:/home/parallels#
root@ubuntu-gnu-linux-24-04-3:/home/parallels#
root@ubuntu-gnu-linux-24-04-3:/home/parallels#
root@ubuntu-gnu-linux-24-04-3:/home/parallels#
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -c 192.168.2.100 -p 5001 -t 30
-----
Client connecting to 192.168.2.100, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[  1] local 192.168.0.2 port 44176 connected with 192.168.2.100 port 5001 (icwnd/mss/irrtt
14/1448/847)
[ ID] Interval      Transfer      Bandwidth
[  1] 0.0000-30.0118 sec  76.1 GBytes  21.8 Gbits/sec
root@ubuntu-gnu-linux-24-04-3:/home/parallels# ping -c 10 192.1658.2.100
ping: 192.1658.2.100: Temporary failure in name resolution
root@ubuntu-gnu-linux-24-04-3:/home/parallels# ping -c 10 192.168.2.100
PING 192.168.2.100 (192.168.2.100) 56(84) bytes of data.

--- 192.168.2.100 ping statistics ---
10 packets transmitted, 0 received, 100% packet loss, time 9410ms
root@ubuntu-gnu-linux-24-04-3:/home/parallels#
```

Figure 4.1.6 – PC1 results during the ICMP flood.

Ctrl+C PC2, then Ctrl+C Server tcpdump

Verification of Firewall - ICMP

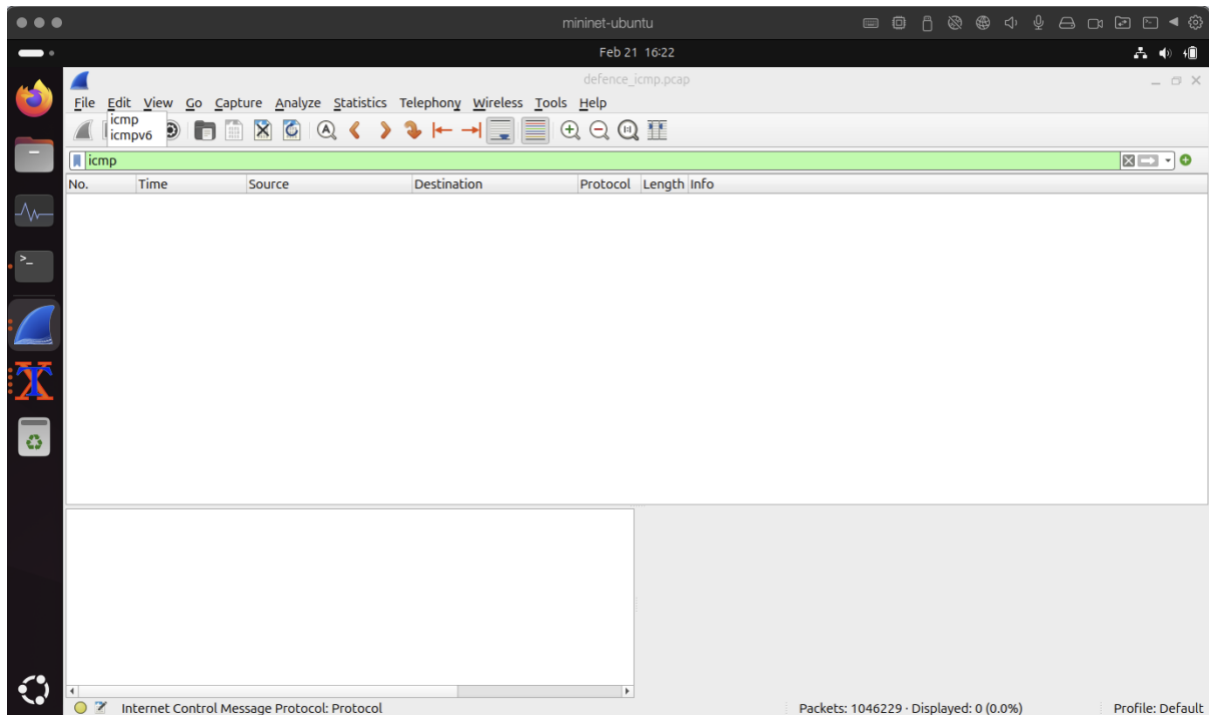


Figure 4.2.1 – Wireshark capture of defence_icmp.pcap filtered for ICMP traffic, showing 0 packets displayed out of 1,046,329 captured (0.0%).

Zero ICMP packets reached the Server. The Router dropped every single one of the flood packets

```
mininet> Router1 iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
Chain FORWARD (policy ACCEPT 14M packets, 569G bytes)
 pkts bytes target    prot opt in     out     source    destination
 180  5040 ACCEPT    1  --  *      *       0.0.0.0/0  0.0.0.0/0    icmp type 8 limit: avg 1/sec burst 4
 49M 1364M DROP      1  --  *      *       0.0.0.0/0  0.0.0.0/0    icmp type 8
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
```

Figure 4.2.2 – Post-attack iptables -L -n -v output on Router1, showing the FORWARD chain counters after the ICMP flood.

DROP showing packets dropped, proving the firewall is working.

Flush Previous Rules

```
mininet> Router1 iptables -F
mininet> Router1 iptables -F
mininet> Router1 iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
Chain FORWARD (policy ACCEPT 14M packets, 569G bytes)
 pkts bytes target    prot opt in     out     source    destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
```

Figure 4.2.3 – Router1 iptables after flushing all rules with iptables -F.

Firewall to Block Router-Forwarded packets from PC3 -> Server

Firewall on the Router

```
mininet> Router1 iptables -A FORWARD -s 192.168.0.4 -d 192.168.2.100 -j DROP
```

Verify the firewall rules

```
mininet> Router1 iptables -L -n -v
```

```
mininet> Router1 iptables -A FORWARD -s 192.168.0.4 -d 192.168.2.100 -j DROP
mininet> Router1 iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source         destination
Chain FORWARD (policy ACCEPT 14M packets, 569G bytes)
 pkts bytes target    prot opt in     out     source         destination
    0    0 DROP      0    -- *      *        192.168.0.4    192.168.2.100
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source         destination
```

Figure 4.3.1 – iptables -L -n -v output on Router1 after adding a FORWARD chain rule to DROP all traffic from 192.168.0.4.

Initiate Attack

Open Xterm

mininet> xterm Server PC1 PC2 PC3

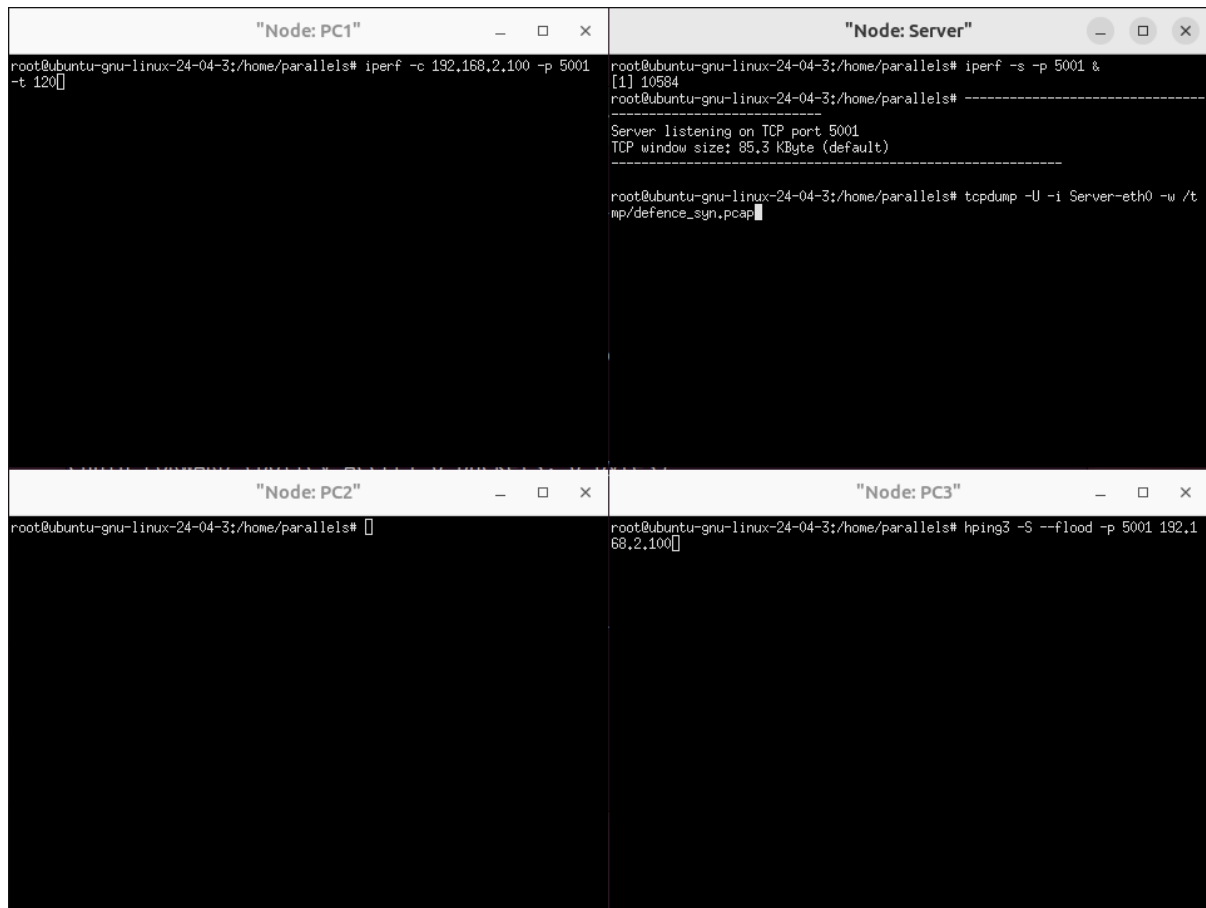
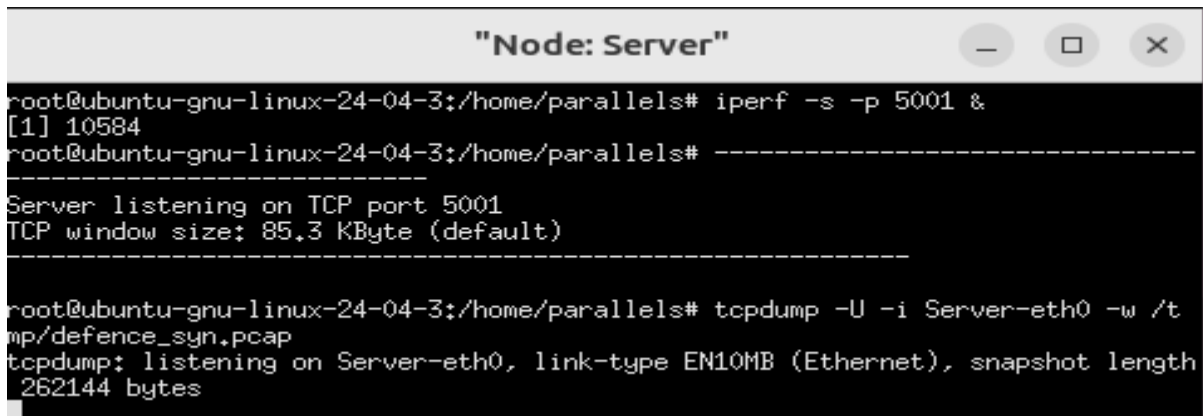


Figure 4.3.2 – All device terminals open.

Server

```
iperf -s -p 5001 &  
tcpdump -U -i Server-eth0 -w /tmp/defence_syn.pcap
```

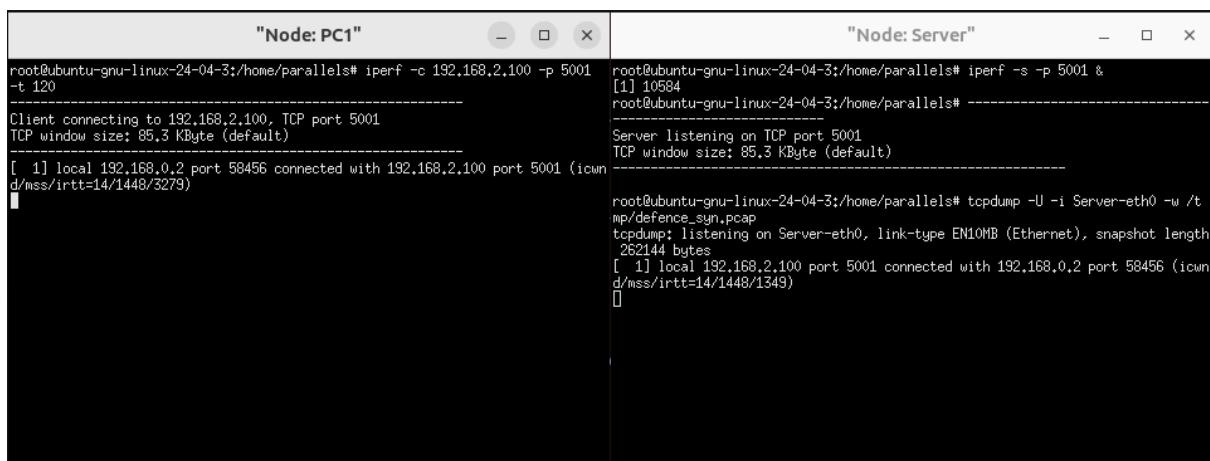


```
"Node: Server"  
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -s -p 5001 &  
[1] 10584  
root@ubuntu-gnu-linux-24-04-3:/home/parallels# -----  
Server listening on TCP port 5001  
TCP window size: 85.3 KByte (default)  
-----  
root@ubuntu-gnu-linux-24-04-3:/home/parallels# tcpdump -U -i Server-eth0 -w /t  
mp/defence_syn.pcap  
tcpdump: listening on Server-eth0, link-type EN10MB (Ethernet), snapshot length  
262144 bytes
```

Figure 4.3.3 – Server node initialisation for the SYN flood defence.

PC1

```
iperf -c 192.168.2.100 -p 5001 -t 120
```

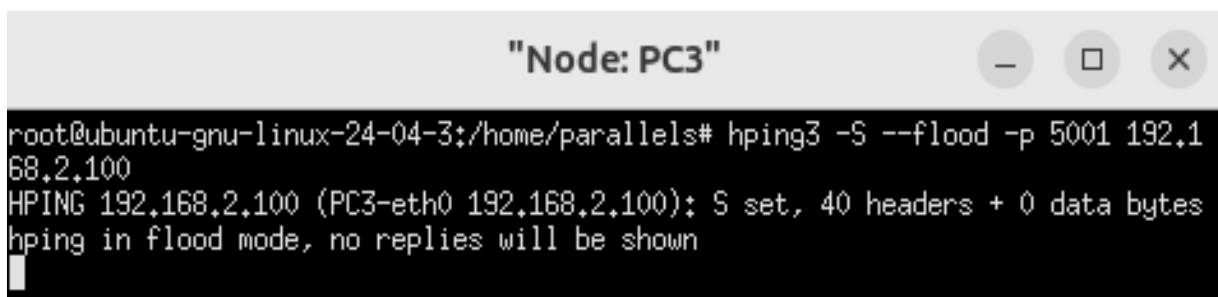


```
"Node: PC1" "Node: Server"  
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -c 192.168.2.100 -p 5001  
-t 120  
Client connecting to 192.168.2.100, TCP port 5001  
TCP window size: 85.3 KByte (default)  
[ 1] local 192.168.0.2 port 58456 connected with 192.168.2.100 port 5001 (icun  
d/mss/irrt=14/1448/3273)  
[ 1] local 192.168.0.2 port 58456 connected with 192.168.2.100 port 5001 (icun  
d/mss/irrt=14/1448/3273)  
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -s -p 5001 &  
[1] 10584  
root@ubuntu-gnu-linux-24-04-3:/home/parallels# -----  
Server listening on TCP port 5001  
TCP window size: 85.3 KByte (default)  
-----  
root@ubuntu-gnu-linux-24-04-3:/home/parallels# tcpdump -U -i Server-eth0 -w /t  
mp/defence_syn.pcap  
tcpdump: listening on Server-eth0, link-type EN10MB (Ethernet), snapshot length  
262144 bytes  
[ 1] local 192.168.2.100 port 5001 connected with 192.168.0.2 port 58456 (icun  
d/mss/irrt=14/1448/1349)
```

Figure 4.3.4 – Side-by-side view of PC1 and Server during the 120-second iperf test. PC1 (192.168.0.2:58456) successfully establishes a TCP connection to the server on port 5001. This is confirmed by the server log showing the accepted connection.

PC3

```
hping3 -S --flood -p 5001 192.168.2.100
```

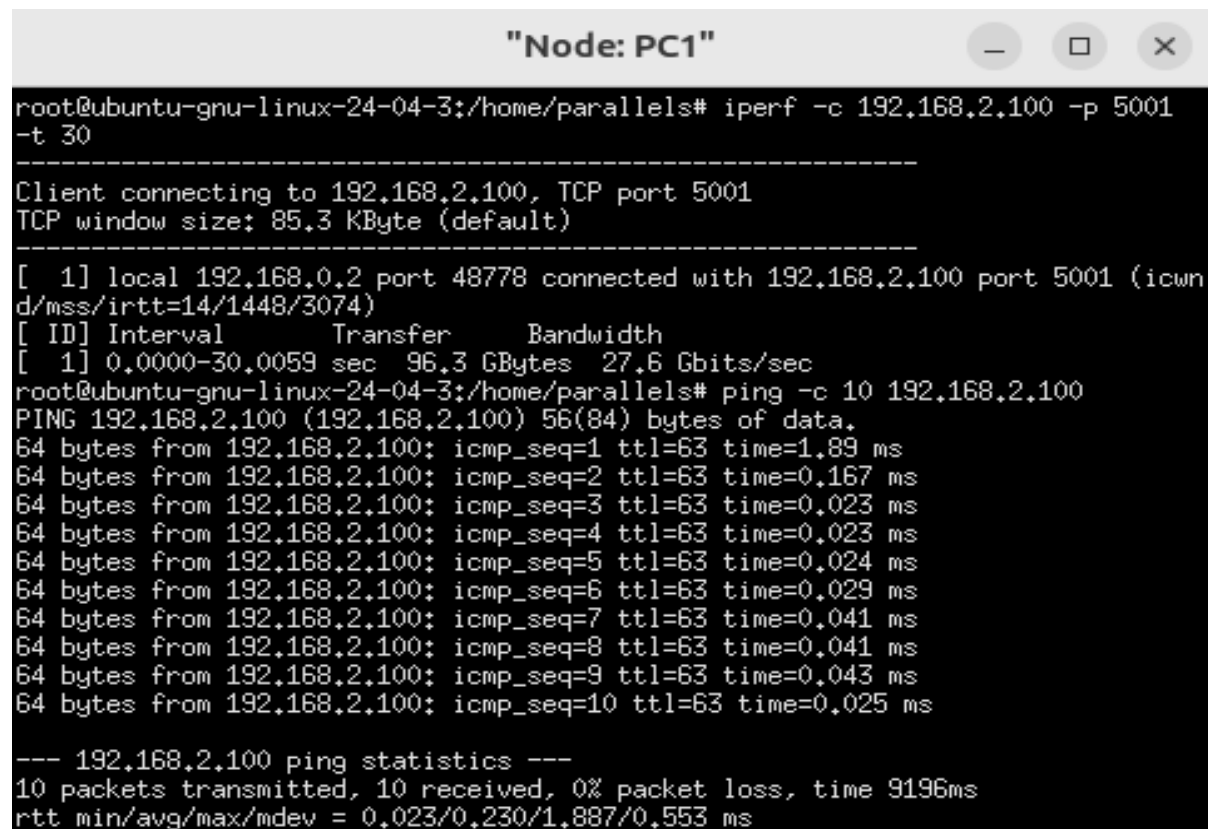


```
"Node: PC3"  
root@ubuntu-gnu-linux-24-04-3:/home/parallels# hping3 -S --flood -p 5001 192.1  
68.2.100  
HPING 192.168.2.100 (PC3-eth0 192.168.2.100): S set, 40 headers + 0 data bytes  
hping in flood mode, no replies will be shown
```

Figure 4.3.5 – PC3 launches a SYN flood attack against the server (192.168.2.100) on port 5001 using hping3 in flood mode.

PC1

```
iperf -c 192.168.2.100 -p 5001 -t 30
ping -c 10 192.168.2.100
```



```
root@ubuntu-gnu-linux-24-04-3:/home/parallels# iperf -c 192.168.2.100 -p 5001
-t 30
-----
Client connecting to 192.168.2.100, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 192.168.0.2 port 48778 connected with 192.168.2.100 port 5001 (icwn
d/mss/irrt=14/1448/3074)
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-30.0059 sec 96.3 GBytes 27.6 Gbits/sec
root@ubuntu-gnu-linux-24-04-3:/home/parallels# ping -c 10 192.168.2.100
PING 192.168.2.100 (192.168.2.100) 56(84) bytes of data:
64 bytes from 192.168.2.100: icmp_seq=1 ttl=63 time=1.89 ms
64 bytes from 192.168.2.100: icmp_seq=2 ttl=63 time=0.167 ms
64 bytes from 192.168.2.100: icmp_seq=3 ttl=63 time=0.023 ms
64 bytes from 192.168.2.100: icmp_seq=4 ttl=63 time=0.023 ms
64 bytes from 192.168.2.100: icmp_seq=5 ttl=63 time=0.024 ms
64 bytes from 192.168.2.100: icmp_seq=6 ttl=63 time=0.029 ms
64 bytes from 192.168.2.100: icmp_seq=7 ttl=63 time=0.041 ms
64 bytes from 192.168.2.100: icmp_seq=8 ttl=63 time=0.041 ms
64 bytes from 192.168.2.100: icmp_seq=9 ttl=63 time=0.043 ms
64 bytes from 192.168.2.100: icmp_seq=10 ttl=63 time=0.025 ms

--- 192.168.2.100 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9196ms
rtt min/avg/max/mdev = 0.023/0.230/1.887/0.553 ms
```

Figure 4.3.6 – PC1 results during the active SYN flood from PC3, with the firewall DROP rule in place.

Ctrl+C PC3, then Ctrl+C Server tcpdump

Analyse Traffic and Verify Firewalls

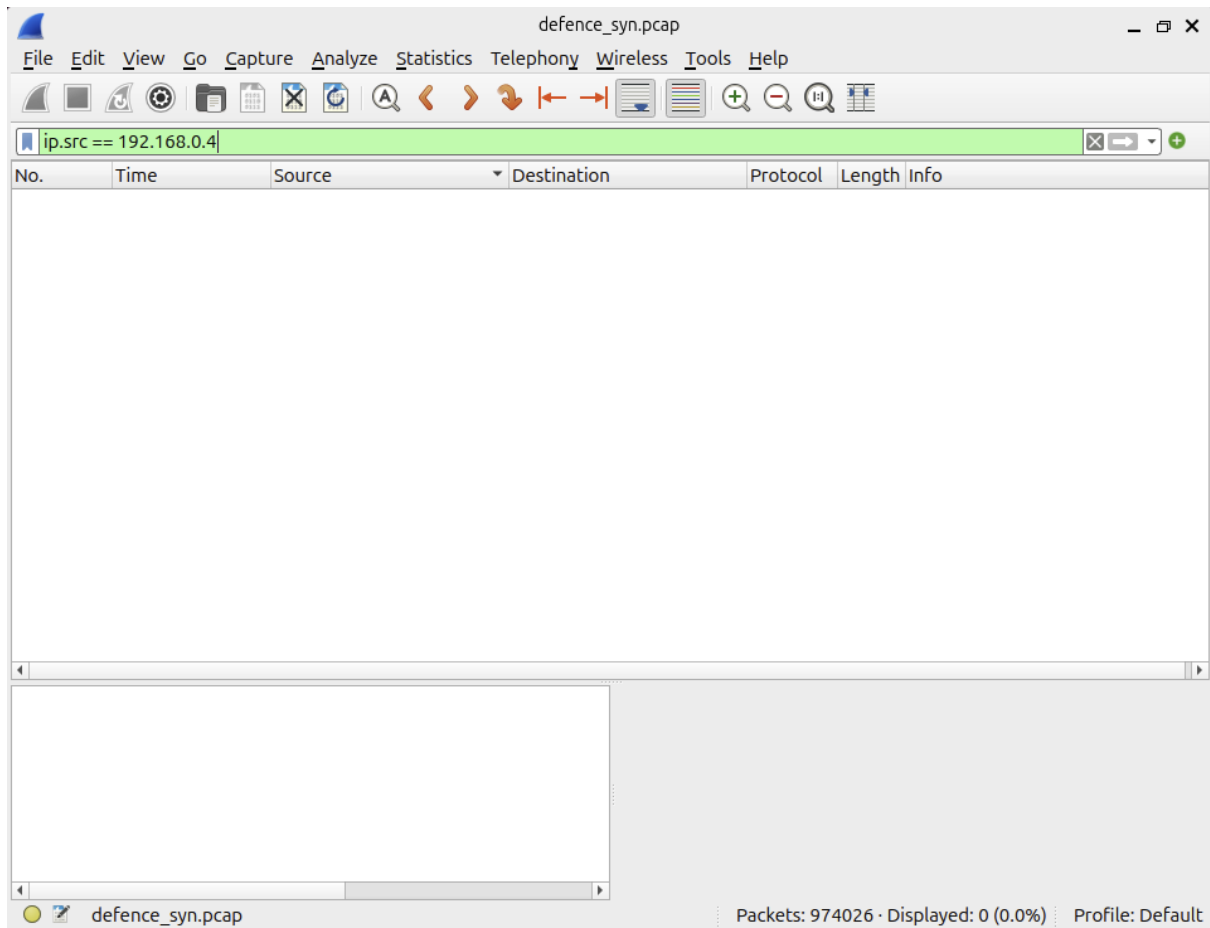


Figure 4.4.1 – Wireshark filtered views show 0 attacker-source packets displayed in the defended capture, supporting the claim that the firewall prevented attacker packets reaching the server interface.

Wireshark: 974,026 packets captured, 0 from the attacker (0.0%) - complete block

```
mininet> Router1 iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in      out     source         destination

Chain FORWARD (policy ACCEPT 13M packets, 574G bytes)
 pkts bytes target    prot opt in      out     source         destination

 56M 2229M DROP      0    --  *        *        192.168.0.4    192.168.2.100
0

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in      out     source         destination
```

Figure 4.4.2 – For SYN flooding, a rule was introduced to drop forwarded traffic from the attacker host to the server. The counters show ~56M packets dropped (2229MB) from 192.168.0.4 → 192.168.2.100.

Comparison Between Levels

Metric	Level 2: Baseline	Level 3: Attack (ICMP / SYN Flood)	Level 4: Defence (Firewall Enabled)
Protocol Distribution	TCP dominant (96.2%), UDP minor (3.8%)	ICMP rises to ~50%, disrupting normal balance	TCP dominant again, ICMP almost eliminated
Traffic Volume (Packets)	~699k packets	~2.1M packets (sharp increase due to attack)	Significant reduction in packets reaching server
Throughput & Performance	Stable (~691 Mbps)	Degraded, unstable with spikes and errors	Restored close to baseline stability
Network Behaviour (Wireshark Graphs)	Smooth, consistent traffic flow	Irregular spikes, bursts, and TCP errors	Stable flow, spikes removed
Security Effectiveness	No protection	Server exposed to DoS (ICMP/SYN floods)	Firewall drops ~49M ICMP and ~56M SYN packets, 0 attacker traffic reaches server

Table 1 – Traffic Analysis and Performance Evaluation Through Comparison of Levels.

Task 5 - Critical evaluation and reflection

This coursework provided a strong practical understanding of how network behaviour evolves during normal operation, under attack scenarios, and in response to defensive interventions.

A key strength of the experiment was the structured comparison across Levels 2, 3, and 4, which enabled clear observation of how ICMP and SYN flooding attacks degrade network performance in terms of throughput, latency, and packet loss. The firewall implementation using iptables demonstrated that targeted rule-based filtering can effectively mitigate these attacks, particularly when applied at a strategic point in the network. The use of a controlled Mininet environment simplifies real-world conditions, in which attackers may employ adaptive techniques to bypass static firewall rules. This highlights that while the implemented defence was effective in this context, it may not generalise fully to real-world networks without additional mechanisms.

From a collaborative perspective, this coursework highlighted the importance of coordination and shared understanding in conducting security experiments. Rather than splitting the work into sections, the group worked collectively on the same system, which improved consistency in the experimental setup and ensured that all members understood both the attack and defence processes.